

Automatic Data Repair: Are We Ready to Deploy?

Wei Ni[*‡1], Xiaoye Miao[*★2], Xiangyu Zhao[‡3], Yangyang Wu[#4], Shuwei Liang[*#5], Jianwei Yin[*#†6]
[*]Center for Data Science, Zhejiang University, Hangzhou, China
[★]The State Key Lab of Brain-Machine Intelligence, Zhejiang University, Hangzhou, China
[‡]School of Data Science, City University of Hong Kong, Hong Kong, China
[#]School of Software Technology, Zhejiang University, Ningbo, China
[†]College of Computer Science, Zhejiang University, Hangzhou, China
{[1]wei.ni, [2]miaoxy, [4]zjuwuyy, [5]lsw5221}@zju.edu.cn [3]xianzhao@cityu.edu.hk [6]zjuyjw@cs.zju.edu.cn

ABSTRACT

Data quality is paramount in today's data-driven world, especially in the era of generative AI. Dirty data with errors and inconsistencies usually leads to flawed insights, unreliable decision-making, and biased or low-quality outputs from generative models. The study of repairing erroneous data has gained significant importance. Existing data repair algorithms differ in information utilization, problem settings, and are tested in limited scenarios. In this paper, we compare and summarize these algorithms with a driven information-based taxonomy. We systematically conduct a comprehensive evaluation of 12 mainstream data repair algorithms on 12 datasets under the settings of various data error rates, error types, and 4 downstream analysis tasks, assessing their error reduction performance with a *novel but practical* metric. We develop an effective and unified repair optimization strategy that substantially benefits the state of the arts. We conclude that, it is always worthy of data repair. The clean data does not determine the upper bound of data analysis performance. We provide valuable guidelines, challenges, and promising directions in the data repair domain. We anticipate this paper enabling researchers and users to well understand and deploy data repair algorithms in practice.

PVLDB Reference Format:
Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, Jianwei Yin. Automatic Data Repair: Are We Ready to Deploy? . PVLDB, 17(10): 2617 - 2630, 2024. doi:10.14778/3675034.3675051

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/WelkinNi/Automatic-Data-Repair.

1 INTRODUCTION

The global application landscape relies heavily on data, while ubiquitously erroneous information involved in data often compromises the data analysis performance, leading to a huge economic cost [17, 43, 49, 84]. Dirty data may trigger the spread of fraud information, inaccurate decision-making, and even legal repercussions [42, 60, 62, 79, 86]. Among the strategies for efficiently mitigating erroneous information and ensuring data quality, *automatic data cleaning* [39, 86] has become a *pivotal* solution, especially in the age of generative artificial intelligence (GAI), where the large volume of high-quality training data for powerful GAI models like ChatGPT [67] and Midjourney [11] currently often resort to *resource-intensive manual* data cleaning [87].

The promising automatic data cleaning task is generally conducted in two consecutive stages: *error detection* and *error repair*. The error detection stage is to identify all wrong data values or rule violation sets, while the error repair stage is to correct these wrong values into latent right ones [2, 56, 57]. Existing error detection studies [38, 57, 65, 71] have achieved obvious advancements, with an average F1 score exceeding 0.85 on real datasets [57, 71]. In contrast, the error repair is more complex and challenging. Various techniques for error repair have been employed, such as machine learning [56, 72, 85], transfer learning [36, 56], boosting algorithm [47] and few-shot learning [56]. However, within the advanced detection results, the *final* data repair performance of current methods still falls short of the expected ideal, with an average F1 score below 0.7 in real scenarios [32, 56, 72]. Thus, it is crucial and urgent to analyze and explore how to deploy *end-to-end* data cleaning approaches with effective *data repair* algorithms in real-life scenarios.

Table 1 compares and summarizes existing surveys of end-to-end data cleaning study. They either analyze the data repair process without experiments [17, 43, 44, 48], or verify the impact of data repair on downstream tasks while overlooking the inclusion of certain important algorithms [1, 51]. The limitations and analysis perspectives come from the following *three* aspects.

L1: Metric shortfalls in evaluation. Fewer data errors typically lead to higher data quality and reduced biases in subsequent analyses, which constitute the *main goals* of the data cleaning task. Prior data cleaning surveys [1, 43, 44, 48, 51] lack an evaluation of *error reduction* in final data repair results, as indicated in Table 1. They often evaluate the data repair results with metrics such as *precision* and *recall*, which are based on the proportion of correctly repaired cells, disregarding the absolute quantity changes. Thus, such metrics result in a *biased* evaluation. For example, suppose there is a dataset that initially encompasses 10 erroneous cells. One data cleaning approach repairs 100 cells, of which 80 are correct (including the initial 10 error cells). Despite 70 of 80 values being identical to their originals, they are still flagged as erroneous values, and replaced by a value within calculation, thereby should be considered as a repaired cell. It means that, the dataset currently contains

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 10 ISSN 2150-8097.
doi:10.14778/3675034.3675051

Table 1: Comparison of previous surveys and ours.

Survey study	Exp. evaluation	Error reduc. evaluation	Error rate		Error type	
			DRE*	DME*	DRE*	DME*
Ilyas et al. [43, 44]	✗	✗	✗	✗	✗	✗
Krishnan et al. [48]	✗	✗	✗	✗	✗	✗
Chu et al. [17]	✗	✗	✗	✗	✗	✗
Li et al. [51]	✓	✗	✗	✗	✗	✓
Abdelaal et al. [1]	✓	✗	✓	✗	✗	✗
Ours	✓	✓	✓	✓	✓	✓

* DRE and DME refer to data repair and downstream model evaluation.

20 erroneous cells, indicating a *doubles* amount of erroneous cells after the repair process. While the precision and recall of this repair both reach as high as 0.8 and 1.0, respectively. The correct repair of more initially right data increases the precision, according to the mediant inequality, but it cannot benefit the data quality. As a result, a higher precision/recall value does not represent a larger degree of error reduction. Furthermore, for distance-based metrics like Jaccard distance or mean squared error (MSE), they only measure the distance between the repaired and clean data, while *ignoring* the distance between the original dirty data and clean data. In a situation where the initial distance between dirty and clean data is 0.1, which then increases to 0.15 after repair. Simply stating the distance as 0.15 fails to directly indicate an increase in data errors. It cannot convey the actual *improvement degree* in data quality, either. Incorporating error reduction degree into evaluations will align repair algorithms more closely with improving data quality, and offer a direct perspective on data quality changes.

L2: Insufficient scenarios exploration in evaluation. The effectiveness of data repair algorithms differs in different scenarios. As depicted in Table 1, existing surveys lack the study of data repair algorithms' capacity under various *error types*, in either the data repair performance evaluation or the assessments of its impact on downstream tasks [1, 17, 44, 48, 51]. Meanwhile, the *error rate* analysis is absent in downstream impact evaluation, even though it is included in repair performance assessments. It results in an inadequate understanding of the proper deployment specific to distinct error types and error rates. It is vital and practical to comprehensively evaluate data repair algorithms with a diverse range of error types and rates in both repair and downstream tasks.

L3: Lacking algorithm deployment guidelines. Blindly deploying data repair algorithms may potentially hurt data quality and task performance. Although existing surveys discuss data repair algorithm performance [17, 43, 44, 48], *three* critical questions regarding algorithm deployment remain unanswered: (i) Is data repair consistently beneficial irrespective of the error rate? (ii) Based on the observation that, the suboptimal performance of data repair primarily results from incorrect repairs, how to develop a practical strategy to mitigate incorrect repairs based on error reduction evaluation? (iii) How to select the proper repair method in different scenarios based on task objectives? The data repair process plays a crucial role in the cleaning task, and addressing these issues will provide practical guidance to deploy data cleaning algorithms.

Therefore, in this survey, we conduct an exhaustive comparative analysis and experimental evaluation of 12 state-of-the-art data repair algorithms. Our main contributions are described below.

- We systematically evaluate *twelve* mainstream and state-of-the-art data repair algorithms based on a driven information-based taxonomy of data repair algorithms, including *cstr-driven* data repair algorithms, *data-driven* ones, and *hybrid-driven* ones. We also state the properly applied scenarios and graphically represent the workflow within each category.
- We define a novel and effective *metric* named *Error Drop Rate* to evaluate the error reduction condition of final data repair results. Surprisingly, in the majority of cases, most data repair algorithms tend to introduce more errors rather than eliminate erroneous information. We propose a *universal* optimization strategy that leverages error detection techniques to prevent the alteration of original right values into wrong ones.
- Extensive experiments on *twelve* real datasets under various error rates/types and data scales demonstrate both the performance of *twelve* data repair algorithms and the impact on *four* common downstream tasks. Particularly, except for five algorithms with publicly available codes, we implement *seven* additional data repair algorithms for comprehensive comparisons. We investigate the effect of *semantic* and *syntactic* data errors (that simulate complex practical scenarios) on the algorithm performance.
- We conclude several interesting findings: *i)* Repair algorithms often *increase* errors. Contrary to expectations, most existing data repair algorithms tend to raise the error rate in the data rather than eliminate it; *ii)* Using the clean data is *not* always the best. Downstream analysis models trained on purely clean data may perform worse than models trained using dirty data. *iii)* Data repair could provide universal benefits for downstream tasks. With proper algorithm selection, data repair boosts downstream tasks in the vast majority of cases; *iv) Semantic* errors pose a more stubborn challenge. Semantic errors within the data are harder to eliminate and have a more negative impact on downstream data analysis than *syntactic* ones, highlighting the need for greater consideration in real-world applications.

In the rest of this paper, Section 2 presents the preliminaries, while the new taxonomy and algorithm analysis are covered in Section 3. Comprehensive evaluations are offered in Section 4. Section 5 discusses current challenges and future directions, followed by related work in Section 6. Finally, Section 7 concludes the study.

2 PRELIMINARIES

In this section, we first introduce the problem definition. We then describe the constraints widely used in automatic repair algorithms.

Data cleaning mainly serves two primary goals: eliminating errors and ensuring data consistency, with the latter being a subgoal of the former. In this survey, we aim to evaluate *error reduction* performance over the final data repair results, as stated below.

Definition 1. **Problem Statement.** Given an instance I of relation R characterized by a set of attributes $Attrs = \{A_1, A_2, \cdots, A_n\}$. Each $t \in I$ comprises a set of cells represented by $Cells[t] = A_i[t]$ corresponding to a distinct attribute in $Attrs$. For each cell c, its unknown true value is denoted by v_c^*, its initial observed value is v_c, and its estimated true value is $\hat{v}_c$. A data repair algorithm aims to make $\hat{v}_c$ equal to v_c^* for all cells c.

This error reduction evaluation defined in *Definition* 1 encompasses correcting all erroneous cells within a dataset. It shares the same definition of *holistic repair*, as studied in most recently proposed data repair algorithms [56, 57, 71, 72].

Expert experience and field knowledge of the data serve as a critical guideline in the data repair process. These guidelines manifest as rules and constraints (cstrs). Rules provide explicit instructions for modifying erroneous values, making data repair a deterministic process [26, 34]. However, real-world scenarios often lack external sources of authoritative information, such as master data or expert insights. This absence poses a significant challenge in employing rules in the data repair process [34]. In contrast, constraints are designed to capture the relationships between specific attributes or values. Constraints have a *wider* range of applications, since professional external information may not always be accessible.

Numerous automatic data repair algorithms are built based on constraints like *functional dependency* (FD) [69] and *denial constraint* (DC) [70], as stated below.

Definition 2. **Functional Dependency.** A functional dependency is a statement of the form $X \rightarrow A$ where $X \subseteq Attrs$ and $A \in Attrs$, denoting that the values of attribute set X uniquely determine the values of attribute A across all tuples in I. X and A are the left-hand and right-hand attributes, respectively.

Definition 3. **Denial Constraint.** A denial constraint φ is a statement of the form $\forall t_\alpha, t_\beta \in I : \neg (p_1 \wedge \cdots \wedge p_m)$ where p_i is in the following form: $t_\alpha.A_x \ \phi \ t_\beta.A_y$ or $t_\alpha.A_x \ \phi \ c$, $A_x, A_y \in Attrs$, c is a constant, and ϕ is a built-in operator, i.e., $\neq$, $=$, $\leq$, $<$, $\geq$, $>$.

Example 1. A sample instance from real-world tax data [4] is presented in Table 2, along with two real FDs and two DCs:

$F_1 : City \rightarrow State$,

$DC_1 : \forall t_1, t_2 \in I : \neg(t_1.City = t_2.City \wedge t_1.State \neq t_2.State)$,

$DC_2 : \forall t_1, t_2 \in I : \neg(t_1.City = t_2.City \wedge t_1.Salary \geq t_2.Salary \wedge t_1.Rate \leq t_2.Rate)$.

The FD F_1 means if the values of $City$ are the same, then the values of $State$ should also be the same. Meanwhile, DC_1 asserts that no two tuples in R can have identical $City$ values paired with differing $State$ values. DC_2 indicates that within a given $City$, a higher $Salary$ should not correspond to a lower $Rate$ of tax. In this case, F_1 and DC_1 are equivalent, expressing the same meaning. The erroneous cells violating these constraints are colored in gray.

An FD in the above form $X \rightarrow A$ can be equally expressed in the following DC format: $\forall t_\alpha, t_\beta \in I$, $\neg(t_\alpha.X_0 = t_\beta.X_0 \wedge \cdots \wedge t_\alpha.X_k = t_\beta.X_k \wedge t_\alpha.A \neq t_\beta.A)$, $X_0, \cdots, X_k \in X$. While DC has greater expressive ability than FD. For instance, the monotonic relationship between values in DC_2 cannot be captured by FD, underscoring the expressivity gap between them. Considering the expressive ability and broader application spectrum, we mainly focus on DC-based data repair algorithms when evaluating the repair solutions adopting constraints.

Based on constraints, to achieve data consistency, previous work [8, 18] also offers other repair definitions, which are significant in algorithm understanding and deployment process:

Definition 4. **Tolerant Repair.** Given a set of constraints Σ over a relation schema R, along with a set of modified constraints Σ', and two instances I and I' of R. (Σ', I') is a tolerant repair of I if I' adheres to the modified constraint set Σ'.

Definition 5. **Consistency Repair.** Given a set of constraints Σ over a relation R, and two instances I and I' of R. I' is a consistency repair of I if I' adheres to the constraint set Σ.

Table 2: Relation of tax.

Tuple	FirstName	LastName	Gender	City	State	Salary	Rate
t_1	Weiming	Posthoff	Female	Vera	Okla.	45,000	6.25
t_2	Weiming	Zongtian	Male	Okemos	Mich.	10,000	3.9
t_3	Shivraj	Alpin	Female	Eastham	Mass.	90,000	5.3
t_4	Yurdaer	Thackray	Female	Eastham	LA.	30,000	5.2

3 DATA REPAIR ALGORITHMS

In this section, we elaborate 12 mainstream data repair algorithms based on a new *information-driven* taxonomy. We present a general and effective strategy to optimize these data repair algorithms.

3.1 Algorithm Taxonomy

Automatic data repair, which typically relies on data and/or constraint information, aims to enhance data quality or improve the performance of downstream tasks.

To offer insights into how they utilize constraints and data according to the driven information, we categorize existing data repair algorithms into three groups, i.e., *constraint(cstr)-driven*, *data-driven*, and *hybrid-driven* methods. The *cstr-driven* methods repair data based on constraints, typically employing *equivalent classes* (which are sets of cells that should satisfy the same constraint conditions). The *data-driven* algorithms only utilize data distribution information (without constraints) to repair data. For *hybrid-driven* algorithms, except for constraints, they also leverage data information not covered in constraints to facilitate data repair.

This taxonomy is useful for providing clear guidelines on using constraints and data, and offering targeted methods for different error types (cstr-driven for semantic, data-driven for syntactic, and hybrid for general errors). It also aligns with varying needs in deployment. Cstr-driven methods for regulated scenarios like healthcare and finance, and adaptable data-driven or hybrid methods for fields like machine learning tasks.

Moreover, there are *three* primary repair objectives within these algorithms, namely *data consistency*, *holistic repair*, and *model performance boost*. Data consistency encompasses *consistency repair*, which is applied to data with strict constraints like, geographic data where city names rely on states, and *tolerant repair*, used when constraints are either outdated or incorrect. *Holistic repair* focuses on maximizing error removal, crucial in fields like cybersecurity or healthcare. Lastly, the aim of *model performance boost* is to refine training data to enhance outcomes of data-driven tasks, such as image classification. Regarding these objectives, cstr-driven algorithms universally aim to achieve consistency repair. Data-driven approaches are for holistic repair and enhancing model performance. While the hybrid-driven methods are oriented toward tolerant repair or holistic repair.

Guided by the taxonomy, we select methods based on the utilization of constraints and data. In *cstr-driven* methods, we examine widely cited Holistic [18] and Nadeef [22] for mainstream graph-based and statistic-based data modeling with DCs. For *data-driven* ones, methods predominantly encompass statistical boosting and machine learning (ML)-based techniques, with BoostClean [47] as the only example for the former, Baran [56] and Scare [85] pioneered for ML applications in data repair. *Hybrid-driven* tools like Relative [8] and Unified [16] are representative in heuristic-based methods, while HoloClean [72] merges constraints with data



Can Foundation Models Wrangle Your Data?

Avanika Narayan
Stanford University
avanika@cs.stanford.edu

Ines Chami
Numbers Station
ines.chami@numbersstation.ai

Laurel Orr
Stanford University
lorr1@cs.stanford.edu

Christopher Ré
Stanford University
chrismre@cs.stanford.edu

ABSTRACT

Foundation Models (FMs) are models trained on large corpora of data that, at very large scale, can generalize to new tasks without any task-specific finetuning. As these models continue to grow in size, innovations continue to push the boundaries of what these models can do on language and image tasks. This paper aims to understand an underexplored area of FMs: classical data tasks like cleaning and integration. As a proof-of-concept, we cast five data cleaning and integration tasks as prompting tasks and evaluate the performance of FMs on these tasks. We find that large FMs generalize and achieve SoTA performance on data cleaning and integration tasks, even though they are not trained for these data tasks. We identify specific research challenges and opportunities that these models present, including challenges with private and domain specific data, and opportunities to make data management systems more accessible to non-experts. We make our code and experiments publicly available at: https://github.com/HazyResearch/fm_data_tasks.

PVLDB Reference Format:
Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. Can Foundation Models Wrangle Your Data?. PVLDB, 16(4): 738 - 746, 2022.
doi:10.14778/3574245.3574258

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/HazyResearch/fm_data_tasks.

1 INTRODUCTION

Foundation Models (FMs) [19] are models trained on broad data that can be adapted to a wide range of downstream tasks. These models have achieved substantial gains across many semantically challenging tasks such as question answering [20], knowledge base construction [79], and information retrieval [39]. As they have scaled to hundreds of billions of parameters (e.g. GPT-3 [20], PaLM [22]), large FMs have demonstrated surprising emergent behaviors and good zero-shot generalization to new tasks (i.e. no task-specific finetuning) on domains vastly different from the data they were pretrained on [22]. These large FMs are often autoregressive language models (e.g. GPT-3 and PaLM) that are trained to predict the next word in large text corpora and can be adapted to new tasks given a simple natural language description of the task (see Figure 1). These breakthrough capabilities have led to a race for building bigger and better models, and innovations continue to push the boundaries of what large FMs can do on a variety of hard *language tasks*.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 4 ISSN 2150-8097.
doi:10.14778/3574245.3574258

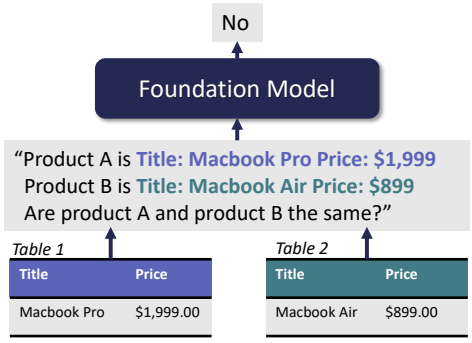


Figure 1: A large FM can address an entity matching task using prompting. Rows are serialized into text and passed to the FM with the question "Are products A and B the same?". The FM then generates a string "Yes" or "No" as the answer.

A natural question that arises is whether these advances can benefit hard classical *data tasks* (e.g. data cleaning and integration). While it is clear that FMs benefit text-intensive tasks, it is not clear whether these models can be applied to data tasks over structured data. The symbols commonly found in structured data (e.g. dates, numbers, alphanumeric codes) are less frequent in natural language text so it is unclear that FMs possess the ability to reason over them. Moreover, since FMs are trained to predict the next word, it is non-obvious that they can work out-of-the-box on complex data tasks. This paper explores the aforementioned question and introduces a new research vision for leveraging FMs for data management, focusing on data cleaning and integration tasks—two keys steps in data-driven enterprise pipelines.

Recently, a large body of research has applied machine learning (ML) [49] and deep learning (DL) [57, 74] methods—namely pretrained language models (PLMs) like BERT [32]—to semantically-complex data tasks. However, these approaches still require a significant amount of engineering effort as they rely on:

- **Task-specific architectures**: Data cleaning and integration encapsulate *many different tasks* such as entity matching [77], schema matching [92], and error detection [41]. Existing approaches, whether they are rule-, ML- or DL-based, vary greatly from one task to the other, often with complex, task-specific architectures. For instance, adapting BERT to data tasks requires architectural changes and finetuning the entire model for each task. This leads to siloed and hard-to-maintain systems.
- **Hard-coded knowledge**: Data tasks often rely on *domain knowledge* (e.g. understanding the relationship between a city and its zip code for data cleaning constraints) and commonsense reasoning. These are usually hard-coded with human-engineered rules or external knowledge bases [24, 84]. Consequently, systems can be brittle and fail to generalize to a diverse set of domains.

- **Labeled data**: ML- and DL-based solutions require copious amounts of hand-labeled data [9]. For instance, PLMs that have achieved state-of-the-art (SoTA) results on data tasks (e.g. Ditto [38]) require a significant amount of task-specific labeled data and fine-tuning to achieve good performance. Labeling data for each task is engineering intensive and adds to the difficulty of maintaining data cleaning and integration systems.

Excitingly, FMs display several useful properties that make them an appealing choice compared to traditional approaches:

- **Task-agnostic architecture**: As a result of their natural language interface, FMs can be applied to a wide-range of tasks. For instance, Figure 1 shows how an entity matching task—which requires identifying whether two table entries refer to the same entity—can be cast as a prompting task. This unifying interface eliminates the need for siloed architectures, in contrast to existing learned approaches where architectures need to be carefully crafted for each task (e.g. task-specific classification layer).
- **Encoded knowledge**: Because FMs are trained on large, generic corpora of data, they contain knowledge about an extensive set of common entities, and thus do not rely on human-engineered rules to acquire knowledge [82].
- **Limited to no labeled data**: FMs can be applied to a breadth of tasks with little to no labeled data (e.g. few-shot and zero-shot). When a FM needs to be fine-tuned, it typically needs dramatically less labeled data to achieve competitive results [46].

Our goal is to better understand if large FMs can be applied to data integration and cleaning tasks. We study the behavior of GPT-3—an early and promising FM. While GPT-3 is already a high quality model, we expect the significant investment in FMs from both academia and industry to lead to more performant and scalable FMs over time. Like many other communities, the data management community stands to benefit from these trends. As such, we aim to understand the advantages and limitations of FMs on data tasks, by focusing on three key questions.

How well do large FMs transfer to data tasks? To answer this, we cast several data tasks as natural language generation tasks (Section 3) and explore whether a single FM can generalize well to these tasks. In Section 4.2, we quantify the zero- and few-shot performance of FMs on five enterprise data tasks: entity matching, error detection, schema matching, data transformation, and data imputation. We find that the largest GPT-3 variant (175B parameters) outperforms SoTA ML-and DL-based approaches on these tasks with few examples. This is particularly surprising since prior approaches are fully-finetuned on task-specific labeled data for these tasks, while GPT-3-175B is simply pretrained to generate text.

What are the caveats in applying FMs to data tasks? In Section 4.3, we unpack the few-shot "prompt tuning" process—serializing tabular data to text, casting data tasks as text generation tasks and constructing demonstrative task examples—for applying FMs to data tasks. We quantify the effects of prompt formatting variations on performance and the differences between manually and randomly selecting task examples. We find that FMs are brittle to differences in prompt formatting and that performance improves when prompts are manually selected versus randomly selected.

What opportunities do FMs present for data tasks and what are the relevant research challenges? Finally, in Section 5, we discuss the potential challenges and related research questions with using FMs in data management pipelines. We discuss the forthcoming shift in how ML systems are built, challenges around updating FM knowledge, and opportunities and considerations pertaining to private, temporal and local data.

We hope that our preliminary exploration will encourage the data management community to explore the effectiveness of FMs for other data tasks and develop techniques to overcome the shortcomings of FMs in this setting.

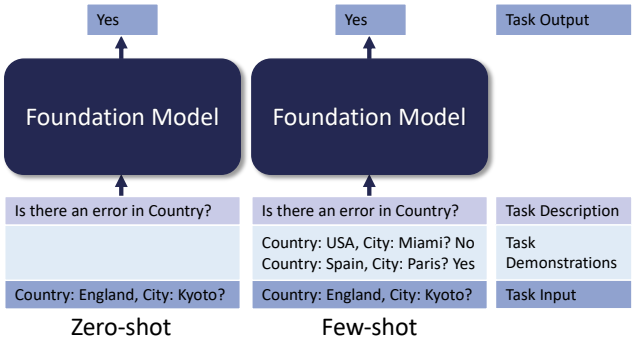


Figure 2: Different ways to use FMs with "in-context" learning [20] on an error detection task. For zero-shot (left), the prompt is the task description and the example to complete. For few-shot (right), the prompt adds task demonstrations.

2 BACKGROUND

We first give some background on the different data tasks considered in this paper and then provide a brief review of FMs.

2.1 Problem Setup

We focus on entity matching (EM), error detection (ED), and data imputation (DI) and describe the setup for these tasks. We denote D, a structured dataset with n entries, such that each entry is represented by a collection of m attribute value pairs: for entry $e_i \in D$ we have $e_i = \{e_{i,j}\}_{1 \le j \le m}$ where for attribute j, $e_{i,j} = \{\texttt{attr}_j, \texttt{val}_j\}$.

Entity Matching The goal of EM is to match entities (real-world objects like people, places and things) across different datasets. Formally, given two structured datasets (D, D') and pairs of entries $e, e' \in D \times D'$, the goal is to predict whether these entries represent the same entity or not. This problem is usually solved as a classification problem, and real-world EM systems are often preceded by blocking heuristics which are used to remove obvious non-matches.

EM has been extensively studied over the past decade (see [77] for a survey) and methods broadly fall into three categories: rule-based, crowd-based [36, 96] and ML/DL-based [49, 74]. Recently, methods relying on PLMs [57] have become SoTA for this task.

Error Detection ED is an important step in data cleaning pipelines. Given an entry e, the goal is to detect attributes j where $\texttt{val}_j$ has an error. The task is framed as a classification problem where the goal is to predict if $\texttt{val}_j$ is correct for a given e.

ED has been studied extensively in both academic and industrial settings [7]. There are a number of successful commercial

offerings including Trifacta [6] and Tamr [5]. Traditionally, ED systems have been heavily reliant on rule-based algorithms which enforce data constraints through functional dependencies or knowledge bases [23, 24, 26]. Additionally, there are statistical-based approaches such as pattern enforcement [24, 45], outlier detection [28], and record deduplication [89] algorithms. Recent efforts have developed SoTA ML models for ED [41].

Data Imputation DI is a critical step for repairing dirty data sources. Given an entry e with missing attribute values $\{\texttt{attr}_j, \texttt{NULL}\}$, the goal of DI is to infer the missing values. The full range of plausible values for the missing value is not known apriori.

Prior works in DI falls into three categories—clustering and/or statistical-based [70], generative model-based [35], ML/DL-based [17, 72], and tabular data pretraining-based approaches [30]—and struggle when needing to impute values not seen in the training set [72].

2.2 Background on Foundation Models

We now give an overview of language FMs, starting from very early, smaller FMs (i.e. PLMs) and moving to large-scale FMs, the latter of which is the focus of this paper.

Pretrained Language Models Pretrained Language Models (PLMs) are neural networks pretrained on large corpora of publicly available text (e.g. web pages). The first breed of PLMs—ELMo [78], BERT [33], RoBERTA [64]—learned the semantics of natural language by predicting masked works during pretraining. These models have on the order of hundreds of millions of parameters. Traditionally, PLMs are adapted to downstream tasks through a task-specific prediction layer (e.g. classification layer) and a task-specific finetuning step wherein all model weights are updated.

Large Autoregressive Language Models In 2020, GPT-3 [20] marked a significant shift in the the ML community. It represented a new class of large-scale language models: autoregressive language models pretrained to predict the next word in a sequence. These models have *billions* of parameters and have been used for language generation tasks such as question answering and summarization. Since the release of GPT-3, bigger and better performing autoregressive language models have been developed [22].

Emergent Behaviors Interestingly, the biggest GPT-3 variant (175B parameters) has the capacity to solve natural language tasks with only a few examples (few-shot prompting), and, in some cases, just a task description (e.g. "Translate French to English"). Unlike traditional finetuning, no model parameters are updated to fit the task. Few-shot prompting has proven to be effective on tasks widely different from the FMs pretraining objective (e.g., code generation [100], Trivia QA [20, 59] and arithmetic [20]). This in-context learning behavior can be thought of as the FM "locating" an already-learned behavior [85]. Smaller models (<10B parameters) typically require some form of task-specific finetuning. We study ways to use smaller FMs on data tasks in the full report [75].

3 FOUNDATION MODELS FOR DATA TASKS

Our goal is to understand whether FMs can benefit data cleaning and integration tasks. The procedure for applying FMs to data tasks requires adapting structured data inputs to textual inputs (Section 3.1), casting data tasks as text generation tasks (Section 3.2) and, for few-shot prompting, constructing demonstrative task examples to help the FM learn new data tasks (Section 3.3).

3.1 Tabular Data Serialization

FMs take text as input and generate text as output. In order to apply these models to data tasks, we first need to convert structured tabular data inputs to text representations.

Given a structured dataset, we first convert an entry to text. Concretely, for entry e, we follow previous work [57] and serialize the attribute value and entry as follows:

$$\texttt{serialize}(e) \coloneqq \texttt{attr}_1 : \texttt{val}_1 \ldots \texttt{attr}_m : \texttt{val}_m$$

If the attribute value is NULL, it is serialized as the empty string. Based on the task and dataset, serialization only happens over a subset of attributes relevant to the task. FMs can be sensitive to the prompt format and the specific serialization used [105]. In Section 4.3 we study the sensitivity of FMs to sub-selection choices.

3.2 Data Tasks as Natural Language Tasks

Next, we need to convert data tasks to text generation tasks. We construct natural language descriptions of each task (i.e. prompts) that use the serialized representations from Section 3.1. The prompts are passed to the FM, whose generated output is the answer to the given task. For entity matching and error detection, the model generates a "Yes" or "No" response,[1] and for imputation it generates the missing value. We now enumerate the prompts for each task.

Entity matching: given two entries (e, e') the template is

```
Product A is serialize(e). Product B is serialize(e').
Are Product A and Product B the same?
```

Data imputation: given an entry e and attribute j to infer, we use

$$\texttt{attr}_1 : \texttt{val}_1 \ldots \texttt{attr}_j \ ?$$

Error detection: given an entry e and attribute j to classify as erroneous, we use

$$\texttt{Is there an error in attr}_j : \texttt{val}_j ?$$

These templates highlight the generality of this framework which could be extended beyond the tasks considered in this paper.

3.3 Task Demonstrations

Task demonstrations can be included in the prompt to help the model learn a new task (see Figure 2 for an error detection example). These demonstrations are used to show the model how the task should be completed (e.g. should it generate Yes/No or a missing value) as well as understand the finer-grained semantics of D. We explore two approaches for selecting task demonstration examples.

Random One approach is to sample random examples from a labeled dataset. However, this approach often causes high variance in FM performance [62]. Moreover, the ordering of examples in the

[1] Interestingly, the model is not constrained to produce a Yes/No answer on the output side, but we find that this happens most of the time. In the rare examples where the model does not predict a Yes/No answer, we use No as the default answer.

DataRinse: Semantic Transforms for Data preparation based on Code Mining

Ibrahim Abdelaziz
IBM Research
Yorktown Heights, New York
ibrahim.abdelaziz1@ibm.com

Julian Dolby
IBM Research
Yorktown Heights, New York
dolby@us.ibm.com

Udayan Khurana
IBM Research
Yorktown Heights, New York
ukhurana@us.ibm.com

Horst Samulowitz
IBM Research
Yorktown Heights, New York
samulowitz@us.ibm.com

Kavitha Srinivas
IBM Research
Yorktown Heights, New York
kavitha.srinivas@ibm.com

ABSTRACT

Data preparation is a crucial first step to any data analysis problem. This task is largely manual, performed by a person familiar with the data domain. *DataRinse* is a system designed to extract relevant transforms from large scale static analysis of repositories of code. Our motivation is that in any large enterprise, multiple personas such as data engineers and data scientists work on similar datasets. However, sharing or re-using that code is not obvious and difficult to execute. In this paper, we demonstrate DataRinse to handle data preparation, such that the system recommends code designed to help with the preparation of a column for data analysis more generally. We show that DataRinse does not simply shard expressions observed in code but also uses analysis to group expressions applied to the same field such that related transforms appear coherently to a user. It is a human-in-the-loop system where the users select relevant code snippets produced by DataRinse to apply on their dataset.

PVLDB Reference Format:
Ibrahim Abdelaziz, Julian Dolby, Udayan Khurana, Horst Samulowitz, and Kavitha Srinivas. DataRinse: Semantic Transforms for Data preparation based on Code Mining. PVLDB, 16(12): 4090 - 4093, 2023.
doi:10.14778/3611540.3611628

1 INTRODUCTION

Enterprises often have many data analysts performing modeling and exploration on the same or similar types of datasets. One of the most time consuming parts of any data analysis step is the preparation of data, such as encoding categorical values, handling ordinals, imputation of missing values, data aggregations, amongst others. This step often requires a significant amount of domain knowledge, and is largely carried out manually by a data engineer, data scientist, analyst or a domain expert. The knowledge, once applied, is often buried in code, and this code tends to be stored in enterprise repositories. Listing 1 shows an example of such a transform, where the data scientist is trying to bin `Age` so that it can be combined with the passenger class attribute to make a combined inference about the wealth and age of the passenger, which likely impacted survival on the Titanic. This snippet of code is adapted from a much larger script that performs multiple operations on the titanic dataset[1]. A key goal of DataRinse is to allow a data scientist re-use of code relevant to the dataset without them having to peruse all the code that manipulates the same dataset.

In DataRinse, we perform this by applying static code analysis on Python scripts. We adopt the path of static analysis compared to dynamic analysis because very often the datasets associated with code in repositories are not available readily making dynamic analysis impractical. It works by finding the transformations to fields of a dataframe, which are then extracted, and converted into an intermediate representation, and this representation is traversed to generate code to help the user transform their dataframe. A key goal of DataRinse is to behave as a recommendation system rather than automating data preparation in one shot. Code generation for data preparation is particularly challenging because: (a) the order in which specific transformations are performed is important; (b) code generation has to be sensitive to the 'groups' of transformations that constitute a single operation conceptually. We illustrate both points with a running example. Listing 1 shows a code snippet from an actual script for preparing the titanic dataset for exploratory analysis. Note that when code is generated, it needs to be aware that a number of transforms are being performed as a 'group', so all the contiguous `Age` transforms bin `Age` into buckets. Furthermore, the transformation of `Age * Pclass` is dependent on that binning operation being performed first; that is, there is a dependency between the first function and the second.

DataRinse performs both grouping of conceptually related operations into functions and encodes dependencies between several functions to offer users functions that can be easily incorporated into their own notebooks. We use control flow from static analysis to perform grouping, and we use control and data flow to understand dependencies between the functions that are generated in DataRinse. Furthermore, to make this consumable to the end user, we display a drop down box in the notebook, so that they can choose which field they want to view cleansing functions for. For each function that is extracted, we also show the URL of the

[1]https://github.com/davified/clean-code-ml/blob/master/notebooks/titanic-original.ipynb

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611628

script that was used to generate that function. Listing 4 shows the output of DataRinse of a code snippet for the Age column, where a single function captures all binning changes to Age, and a second separate function shows the creation of a new column using it. The user has a number of such functions per column that can be readily applied to their dataset if they deem it applicable, and the ordering of transformations is encapsulated in the code generated such that a call to transform Age is performed before the use of it in expressions such as `Age * Pclass`, as shown. All the generated functions are dynamically evaluated on the dataframe to ensure that any functions that are not compile-able are filtered out. Similarly, the generated functions that do not result in a change in the values of the dataframe are also eliminated.

We note that ChatBots such as ChatGPT[2] can appear to provide similar functionality for such tasks. There are however key differences between those systems and DataRinse. While ChatGPT generates some useful data operations, it has the following characteristics that are different from DataRinse: (a) It has non-determinism in its generation, it can sometimes find cleansing operations, but produces different results on each run; a well-known problem for ChatBots (see Listings 2, 3 for two different code snippets produced for the same dataset (b) It does not follow basic data operation rules such as being consistent across train and test, and it is not comprehensive - `Fare` is handled only in one snippet. (c) There is no attribution to the source that these are drawn from, (d) It is not focused - the snippets were drawn from a much larger script, where the system produced imports, read CSV files, etc. (e) It can only work off its training dataset - DataRinse can for instance run over any scripts in any repository at any desired frequency (e.g., on every commit or daily) requiring negligible computational resources; ChatBots have to be retrained on enterprise specific code.

2 SYSTEM OVERVIEW

Figure 1 describes the overall DataRinse system for data preparation. Given a new dataset, the system queries code repositories such as GitHub with the table's metadata, specifically the table and column names. One current limitation of DataRinse is that it will only select scripts that match column names and table names - this is a limitation we plan to address in future work through more flexible column-to-concept mappers [7]. The analysis framework is an open source project that we created[3] which performs inter-procedural static analysis to create control flow and data flow graphs for Python scripts [1]. The scope of the analysis is the entire notebook, so dataflow and control flow can be used to order the cleansing operations. We perform interprocedural analysis to trace both data and control flow, as illustrated in Figure 2. The chalky lines for `read_df` and `prep_df` denote function calls, and dataflow follows them, e.g `"titantic_train.csv"` flows to `fl_name`. After analysis, the code generation component for cleansing starts by isolating reads, writes and updates to pandas dataframe. It uses dataflow information to determine which expressions are relevant, e.g. dataflow connects the read of a pandas dataframe at the top of Figure 2 to the update of `dataset` at the bottom.

[2]https://openai.com/blog/chatgpt
[3]https://github.com/wala/graph4code/tree/master/semForms

```python
import pandas as pd

def read_df(fl_name):
  return pd.read_csv(fl_name)

def prep_df(combine):
    for dataset in combine:
        dataset.loc[ dataset['Age'] <= 16, 'Age'] = 0
        dataset.loc[(dataset['Age'] > 16) &
        ↪  (dataset['Age'] <= 32), 'Age'] = 1
        dataset.loc[(dataset['Age'] > 32) &
        ↪  (dataset['Age'] <= 48), 'Age'] = 2
        dataset.loc[(dataset['Age'] > 48) &
        ↪  (dataset['Age'] <= 64), 'Age'] = 3

def main():
    train_df = read_df("titanic_train.csv")
    test_df =  read_df("titanic_test.csv")
    combine = [train_df,test_df]
    prep_df(combine)
    for dataset in combine:
        dataset['Age*Class'] = dataset.Age *
        ↪  dataset.Pclass
```

Listing 1: Example data preparation code, handling the titanic dataset

```python
train_df = train_df.drop(['PassengerId', 'Name',
↪  'Ticket', 'Cabin'], axis=1)
train_df['Age'].fillna( train_df['Age'].median(),
↪  inplace=True)
train_df['Embarked'].fillna(
↪  train_df['Embarked'].mode()[0], inplace=True)
train_df['FamilySize'] = train_df['SibSp'] +
↪  train_df['Parch'] + 1
train_df['IsAlone'] = 1
train_df['IsAlone'].loc[train_df['FamilySize'] > 1]
↪  = 0
train_df['Sex'] = train_df['Sex'].map({'female': 0,
↪  'male': 1}).astype(int)
```

Listing 2: Code snippets from ChatGPT, when prompted to prepare the titanic dataset for analysis, first attempt

The code generation component assembles related code into multiple functions, organized by the fields that they change, as well as records any dependencies in the application of the functions by encoding calls to relevant functions. In Pandas, calls on a dataframe can sometimes mutate the dataframe (e.g. `drop()`), and sometimes not (e.g. `isnull()`). Because there is no way of knowing if the values of a dataframe are actually modified by a call, the filtering component dynamically evaluates the functions returned by the code generation module in the client notebook to filter out functions that do not transform the data frame. Extracted functions across

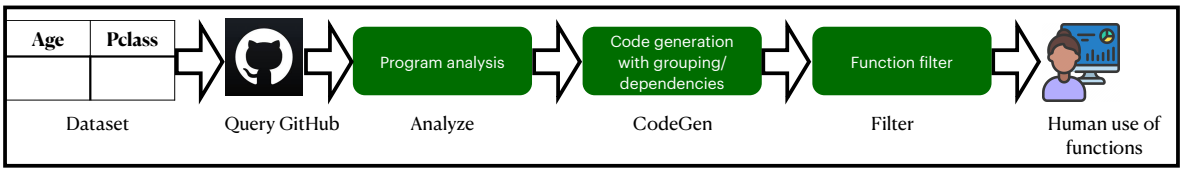


Figure 1: Overview of DataRinse

```
train_df.drop(['PassengerId', 'Name', 'Ticket',
  ↪  'Cabin'], axis=1, inplace=True)
test_df.drop(['Name', 'Ticket', 'Cabin'], axis=1,
  ↪  inplace=True)
train_df['Age'].fillna( train_df['Age'].median(),
  ↪  inplace=True)
test_df['Age'].fillna(test_df['Age'].median(),
  ↪  inplace=True)
test_df['Fare'].fillna( test_df['Fare'].median(),
  ↪  inplace=True)
```

Listing 3: Code snippets from ChatGPT, when prompted to prepare the titanic dataset for analysis, second attempt

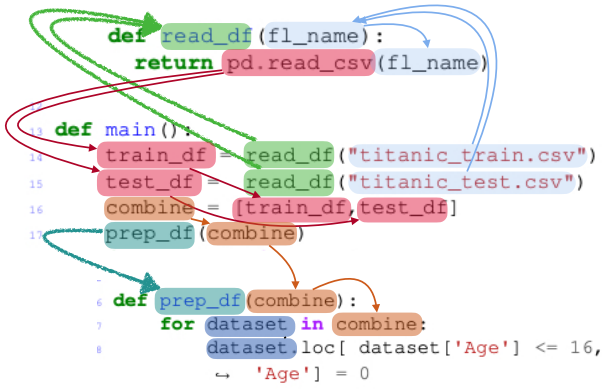


Figure 2: Analysis on listing 1

multiple scripts are then grouped by field, and each function for a given field encapsulates the extraction from a single script. The user can choose which functions they would like to look at, and explore what they would like to add to their own code through an interactive interface.

3 DEMONSTRATION

We show in Listing 4 an example of how DataRinse recommends data transforms is used for the example code shown in 1, which is a simplified version of the original code in[4], for the titanic dataset[5]. The UI is structured as a Jupyter notebook where the user loads

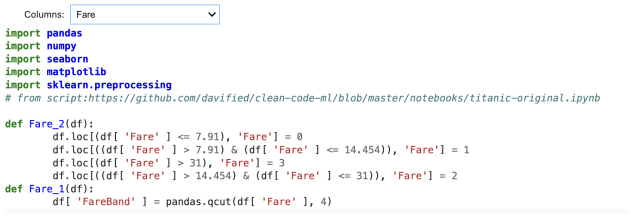


Figure 3: Display of expressions by columns

a dataset. The dataset name and columns are fed to GitHub to retrieve scripts that may be cleansing the same dataset. The scripts are analyzed, and code generation is performed for each field in that is manipulated in the scripts. Any dependencies in manipulations are captured by the code generation as well, as shown in Listing 4. A second example is shown in Listing 5 for the UCI heart disease prediction dataset[6]. Notice here that the order of operations in the original script[7] are maintained, which involves setting missing values in the resting ECG results to `normal` before using a `LabelEncoder` to encode all the values.

In general, there may be too many data manipulation operations across scripts. Since the generated code relies on an IR, obvious duplicates are eliminated at creation time across scripts. Scripts are also ordered by the ones that have the most number of functions across fields, because those are the most comprehensive. The code suggestions are organized by columns using a dropdown as shown in Figure 3. At the conference, we will allow the audience to select an available dataset, and explore the code segments provided by the system. We will also work with the audience to use a dataset of their choice.

4 EXPERIMENTS

To assess the generality of DataRinse as a tool, we applied it to 1,589 Python notebooks from GitHub that were converted into Python scripts for analysis. Table 1 shows some statistics of running DataRinse over these scripts. Approximately 11% (182) of the 1,589 scripts had failures because the code could not be parsed or was not actually Python code; so that left us with 1407 scripts. Analysis ran successfully on most of those scripts (1,406); but after removal of duplicate scripts, we had 1,142 scripts. Of the 1,142 scripts that were successfully analyzed, 55% yielded at least one cleansing function. The total number of CSV files mentioned in these scripts from which cleansing functions were extracted was 2,262, but we note that this is likely an overestimate since many scripts load train and test splits

[4]https://github.com/davified/clean-code-ml/blob/master/notebooks/titanic-original.ipynb
[5]https://www.kaggle.com/competitions/titanic/data

[6]https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data
[7]https://www.kaggle.com/code/achintyak/decision-tree

Lingua Manga : A Generic Large Language Model Centric System for Data Curation

Zui Chen
IIIS, Tsinghua University
Beijing, China
chenzui19@mails.tsinghua.edu.cn

Lei Cao
U of Arizona/MIT
Tucson, Arizona
lcao@csail.mit.edu

Sam Madden
MIT
Cambridge, Massachusetts
madden@csail.mit.edu

ABSTRACT

Data curation is a wide-ranging area which contains many critical but time-consuming data processing tasks. However, the diversity of such tasks makes it challenging to develop a general-purpose data curation system. To address this issue, we present Lingua Manga , a user-friendly and versatile system that utilizes pre-trained large language models. Lingua Manga offers automatic optimization for achieving high performance and label efficiency while facilitating flexible and rapid development. Through three example applications with distinct objectives and users of varying levels of technical proficiency, we demonstrate that Lingua Manga can effectively assist both skilled programmers and low-code or even no-code users in addressing data curation challenges.

PVLDB Reference Format:
Zui Chen, Lei Cao, and Sam Madden. Lingua Manga : A Generic Large Language Model Centric System for Data Curation. PVLDB, 16(12): 4074 - 4077, 2023.
doi:10.14778/3611540.3611624

1 INTRODUCTION

In the era of big data, organizations are collecting vast amounts of data from various sources. However, the data is often messy, incomplete, or contains errors. To effectively perform data analytics or any other applications, it is often necessary to engage in a data curation process [8, 15], which includes tasks like data discovery, integration, and cleaning. Nonetheless, devising a data curation solution for a particular scenario can be a cumbersome and time-consuming process. It entails extensive communication of requirements, a collaboration between domain experts and programmers, multiple rounds of debugging and testing, and active maintenance to accommodate new use cases. This underscores the need for a *generic* system that enables users to efficiently and effectively address diverse data curation challenges and apply solutions to various applications.

However, building a system that addresses diverse data curation problems and applications is challenging. Data curation involves a range of distinct tasks, such as data discovery through table search, schema matching, entity resolution during data integration, and data imputation in data cleaning. Moreover, in real applications, various extra tasks are often involved in the data curation processes,

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611624

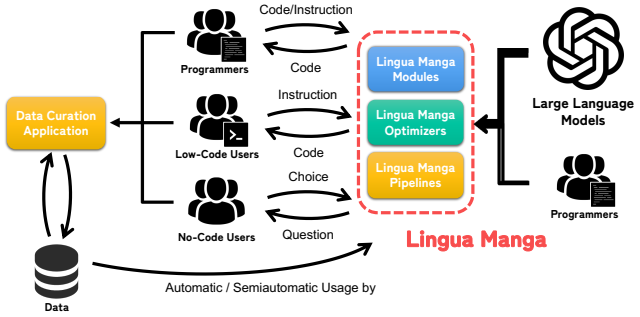


Figure 1: An illustration of the Lingua Manga System.

such as event extraction, name extraction, anomaly detection, data summarization, visualization, table semantic parsing, etc. Worst yet, applications in different domains have diverse requirements. Effectively resolving data curation problems often relies heavily on users' domain knowledge, making developing a *general-purpose* solution improbable. For example, entity resolution over *books* focuses on text understanding, while entity resolution over *images* should rather pay special attention to image processing. As a result, existing data curation systems typically focus on only a few tasks on specific data formats. For example, Data Tamer [8] solves schema matching and entity resolution problems; ZenCrowd [4] targets entity linking; while CrowdDB [7] addresses fuzzy SQL semantics.

Recent advances in Large Language Models (LLMs) have shed light on this highly challenging yet critical data curation area. Pre-trained LLMs such as GPT-3 [10], Codex [6], ChatGPT [17], and LLaMA [5] have not only shown the ability to understand human needs and instructions but also to generate code like a programmer. Such ability could be leveraged to solve specific data curation problems like data imputation or entity resolution [2].

However, the current use of LLMs and LLM plugins is still far from the ultimate solution. Repeatedly calling LLM services can be costly and may lead to data privacy issues. For instance, consider a large e-commerce organization with thousands of documents and tables containing billions of rows – it would be neither affordable nor secure to let an LLM access the entire data lake to complete the tasks. LLMs tend to be less effective without access to enterprise data due to the gap between the enterprise data and the public data LLMs trained on. Thus, there is an urgent need to harness the power of LLMs systematically, efficiently, and securely. Properly integrating LLMs and traditional data curation methods could be a promising path toward a general-purpose data curation system.

In this demonstration, we present Lingua Manga , an LLM-centric system that enables the swift development of effective yet efficient data curation solutions. It benefits programmers as well as low-code or even no-code users. Specifically, Lingua Manga shows the following key properties:

• **User-friendly**: Users can quickly develop a data curation solution by utilizing the templates, built-in modules, and, most importantly, by effectively interacting with LLMs through Lingua Manga as a middleware.

• **Flexible**: Users can effortlessly communicate with Lingua Manga through natural language (NL) to inject domain knowledge or instructions into the data curation solution such that it can meet the needs of the specific applications or domains.

• **Intelligent**: Lingua Manga automatically optimizes the solution to fix errors, improves performance, continuously learns from the data, etc.

• **Highly Performant**: Lingua Manga minimizes the frequency of calling the LLM service, which incurs heavy computation and monetary costs, thus resulting in an economical, efficient, and privacy-preserving solution.

• **Label Efficient**: with our system, users can develop a data curation solution with no or only a few labeled examples from the specific application while still achieving accuracy comparable to the SOTA ML-based methods trained with thousands of labels.

2 PRELIMINARIES: PROMPT-TUNING AND CODE GENERATION IN LLMS

Starting from GPT-3 [10], LLMs' size and computational cost have grown so much that users can no longer afford to fine-tune them for every downstream task. In the meantime, LLMs have also gained significant knowledge through training on large-scale corpora, leading to the emergence of a new paradigm known as prompt-tuning [9]. Using prompt-tuning, users can generalize LLMs to new tasks and data domains without modifying the LLM itself. This is achieved by incorporating prompts, such as descriptions, tags, and learnable components, into the input data.

Moreover, cutting-edge LLMs like ChatGPT [17] have demonstrated their effectiveness in interacting with humans using natural-language-based prompts [3]. These prompts are interpretable by humans. For instance, a straightforward prompt like "Please determine if the following entities are equivalent" can enable LLMs to carry out entity resolution. This capability is especially useful in Lingua Manga since it facilitates LLMs to interactively execute functions vaguely specified by users in natural language.

Furthermore, prompts like "Please write a code for entity resolution" trigger LLMs to generate custom software modules via code generation. GPT-3 [10], ChatGPT [17], Codex, and Copilot [6] are the mainstream LLMs that support code generation. However, existing methods that use these models to generate code have limitations [19]. They are typically inadequate for producing code with complex logic and building large-scale software. This calls for a new methodology to effectively utilize LLM Generated Code (LLMGC).

3 LINGUA MANGA

Lingua Manga comprises a Domain-Specific Language (DSL), a Compiler, an Optimizer, and a set of templates.

Lingua Manga is a **user-friendly** workflow system that enables users to quickly build data curation solutions by composing pipelines of logical operators. The system features a DSL to simplify the workflow-building process.

Lingua Manga is **automatic**. Like a relational database, it auto-compiles each logical operator into a physical, executable *module*.

Lingua Manga is **extensible**. Lingua Manga allows the programmers to implement their own physical modules, which can be seamlessly plugged into the system as long as they follow the standard interfaces of Lingua Manga .

Lingua Manga is **flexible**. To meet the specific need of different applications, the Lingua Manga optimizer interacts with users via LLMs to generate customized physical modules.

Lingua Manga is **intelligent**. The Lingua Manga optimizer uses LLMs to enhance the modules' capability.

In addition, to further facilitate users in building pipelines, Lingua Manga offers templates, essentially groups of pre-built pipelines. Rather than creating a pipeline from scratch, Lingua Manga allows users to start with a pre-defined, well-optimized pipeline that the target application can directly use.

The fundamental observation underlying this research is that while current LLMs may not yet be able to produce exceedingly intricate and large software systems, they are proficient at generating modular code on a smaller scale. Moreover, LLMs possess learning and logical reasoning ability that enable them to complete data curation tasks directly, indicating that LLMs could accelerate the process of developing a data curation solution by automating parts of it and, thus, reduce the workload of software developers.

Next, we discuss the modules and the Lingua Manga optimizer, which are the critical components of Lingua Manga .

3.1 Modules in Lingua Manga

A module is a function $f : X \rightarrow Y$ that takes X as input and returns Y. Modules are usually viewed as black boxes, and their behavior is determined by their expected output. In Lingua Manga , modules can be classified into four types:

• **Custom Module**: As a basic module, a custom module is implemented with manually written code. It can be created by users with programming skills or provided by Lingua Manga as a default built-in module.

• **LLM Module**: An LLM itself can be a module. An LLM module tends to be more powerful than a traditional module because of the common sense knowledge and logical inference abilities possessed by an LLM. However, an LLM module requires a good task description as input; and LLM outputs typically need proper validation, as textual responses generated by the LLMs could be diverse and unstable. For instance, numerical validation that ensures the LLM's output is within a reasonable value range can identify and correct potential errors in LLM output.

• **LLMGC Module**: An LLM can dynamically generate code to implement an LLMGC module, replacing the role of programmers. Lingua Manga allows LLMGC to call other modules in the system or use external tools such as a calculator or another pre-trained LLM. Although an LLMGC module might be less expressive than custom modules written by skilled programmers, it enables no-code or low-code users to implement or customize a module effortlessly.

• **Decorated Module**: As the most advanced module in Lingua Manga , a decorated module can comprise multiple basic modules and be enhanced by the optimizer that leverages the common sense knowledge encoded in the LLMs and their learning ability.

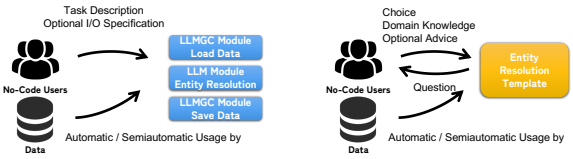
3.2 Lingua Manga Optimizer

The optimizer in Lingua Manga can improve the performance of physical modules within a pipeline by enhancing their efficiency and effectiveness, e.g., avoiding repeatedly calling the LLM service to minimize the execution costs and the potential data privacy leak. Unlike conventional database optimizers, the optimizer in Lingua Manga is modularized and can be selectively composed by users to serve specific applications best. The key modules in Lingua Manga include:

- **Validator**: It checks whether the target module behaves correctly on a few example test cases. It then uses the failed test cases to trigger the LLM to improve the target module and fix the errors. Specifically, the validator first calls an LLM to generate the suggestion by reading the code and the failure cases. Then, the code, failure cases, and the generated suggestion are sent to another LLM to generate a new version of the code. This validation cycle repeats until either all test cases are executed successfully, or a timeout ensues, leading to a re-generation of the LLMGC module until an additional timeout.
- **Simulator**: A simulator automatically generates a more efficient and equally effective alternative to a given module that already functions well. For example, a module that frequently calls LLM service can be expensive. Potentially, a supervised learning counterpart could replace it. This is because LLMs are versatile, while a data curation task typically only needs part of its functionality. It is thus possible to produce a module proficient in simulating the specific LLM functionality required by a task.

  Because each module is treated as a black-box function, an ML-based simulator can replicate the target module through supervised learning. The target module will function as intended during initialization, and a control logic will decide when the simulated version should take over, such as after achieving the desired accuracy or reaching a certain level of confidence.

  A simulator could improve accuracy as well. At first sight, this is counter-intuitive. Like the teacher-student model in machine learning, traditionally, the student model, which learns from a teacher model, is expected to be upper bounded by the teacher model on performance. However, studies have shown that self-training with filters [12, 16, 18, 20] can produce a superior student model because of better generalization. Moreover, the simulator will continuously monitor the real data flow in the original pipeline. It can thus constantly learn to adapt to the data distribution, potentially outperforming the original static module.
- **Connector**: Concerning efficiency and data privacy, it is crucial for applications to reduce the amount of data exposed to LLMs, while still ensuring the effectiveness. For instance, in an NL2SQL task, granting LLMs access to the entire table contents is neither secure nor cost-effective. However, relying solely on the table schema is not adequate to generate accurate predicates. To address this issue, a locally-running connector can be employed to manage the selective data upload to LLMs. A pre-defined connector for tabular data enables LLMs to execute SQL commands in local databases and obtain the resulting data while ensuring that the execution is limited to the queries specified by the user. In multi-modal data scenarios, appropriate pre-defined connectors are provided, such as connectors designed for handling extensive textual data.



(a) Custom pipeline. **(b) Builit-in template.**

Figure 2: An illustration of two possible Entity Resolution workflows using Lingua Manga .

4 DEMONSTRATION

We use three applications to demonstrate Lingua Manga . Fig. 5 shows its user interfaces with name extraction as an example task.

4.1 Entity Resolution: Effortless to the Novices

Consider the scenario where a technical novice wants to perform entity resolution on a dataset but lacks programming experience.

Lingua Manga effectively fulfills these requirements by providing a user-friendly solution. To begin, users can easily search for existing templates within the system. If none are available, they can create a simple pipeline that includes data loading, entity resolution, and data-saving operators, as depicted in Figure 2a. It is worth noting that the user does not have to write any code. Instead, they can simply describe the task to the LLMs using the suggested prompt templates and provide optional input and output specifications through examples. The Lingua Manga optimizer will then improve the performance automatically.

We evaluated the F1 scores of the solution produced by Lingua Manga on three entity resolution datasets: BeerAdvo-RateBeer, Fodors-Zagats, and iTunes-Amazon [1]. As shown in Table 1[1], it significantly outperforms the previous LLM-based method [2], while achieving performance comparable to supervised learning algorithms trained with hundreds or even thousands of labeled examples [13].

Table 1: Quantitative Experiment on Entity Resolution.

Dataset	Magellan [1]	Ditto [13]	FMs [2]	Lingua Manga
BeerAdvo-RateBeer	78.8	94.37	78.6	89.66
Fodors-Zagats	100.0	100.00	87.2	95.65
iTunes-Amazon	91.2	97.06	65.9	92.00

4.2 Name Extraction: Flexible for the Adepts

Consider a scenario where a domain expert, as a low-code user with basic programming skills, aims to get a high-accuracy and efficient name extraction solution, i.e., find all person names in a text passage. In this case, the domain expert understands that name extraction typically involves three steps: tokenization, noun-phrase extraction, and tagging. As a result, the domain expert can create a pipeline of three individual operators (ignoring data loading and saving for ease of presentation), as shown in Fig. 3.

The domain expert recognizes that tagging is a complex process and is often the performance bottleneck. Therefore, he or she chooses to utilize the LLM module within Lingua Manga along with an example-based validator. The other two operators are relatively simple and thus are realized as LLMGC modules – using LLM to generate the code. Suppose the resulting LLMGC modules, such as the noun-phrase extraction module, are not precise enough. The

[1]The numbers of the baselines are obtained from [2].

Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph

Yiming Lin*
University of California, Irvine
yiminl18@uci.edu

Yeye He
Microsoft Research
yeyehe@microsoft.com

Surajit Chaudhuri
Microsoft Research
surajitc@microsoft.com

ABSTRACT

Business Intelligence (BI) is crucial in modern enterprises and billion-dollar business. Traditionally, technical experts like database administrators would manually prepare BI-models (e.g., in star or snowflake schemas) that join tables in data warehouses, before less-technical business users can run analytics using end-user dashboarding tools. However, the popularity of self-service BI (e.g., Tableau and Power-BI) in recent years creates a strong demand for less technical end-users to build BI-models themselves.

We develop an Auto-BI system that can accurately predict BI models given a set of input tables, using a principled graph-based optimization problem we propose called *k-Min-Cost-Arborescence* (k-MCA), which holistically considers both local join prediction and global schema-graph structures, leveraging a graph-theoretical structure called *arborescence*. While we prove k-MCA is intractable and inapproximate in general, we develop novel algorithms that can solve k-MCA optimally, which is shown to be efficient in practice with sub-second latency and can scale to the largest BI-models we encounter (with close to 100 tables).

Auto-BI is rigorously evaluated on a unique dataset with over 100K real BI models we harvested, as well as on 4 popular TPC benchmarks. It is shown to be both efficient and accurate, achieving over 0.9 F1-score on both real and synthetic benchmarks.

PVLDB Reference Format:
Yiming Lin, Yeye He, and Surajit Chaudhuri. Auto-BI: Automatically Build BI-Models
Leveraging Local Join Prediction and Global Schema Graph. PVLDB, 16(10): 2578 - 2590, 2023.
doi:10.14778/3603581.3603596

1 INTRODUCTION

Business Intelligence (BI) is increasingly important in modern enterprises for data-driven decision making, and has grown into a multi-billion dollar business [25]. In traditional BI settings, database administrators (DBAs) typically need to manually prepare BI-models (table schemas and join relationships) in data warehouses, so that less-technical business users can perform ad-hoc analysis using tools like dashboards [16].

In recent years, in a growing trend called "*self-service BI*" [26] that is popularized by vendors like Tableau [11] and Power-BI [9], less-technical business users are increasingly expected to set up and perform BI analysis themselves, without relying on DBAs or central IT. The goal is to democratize BI, so that business users can make agile data-driven decisions themselves, without depending on technical users.

Building BI models: still a pain point. At a high level, there are two main steps in BI project: (1) building BI models, and (2) performing ad-hoc analysis by querying against BI models. While querying BI-models was made simple by vendors like Tableau and Power-BI (through intuitive user interfaces and dashboards) [28, 38], the first step of building "BI-models", a prerequisite for ad-hoc analysis, remains a key pain point for non-technical users.

In the context of self-service BI tools like Tableau and Power-BI, "BI-modeling" refers to the process of preparing raw data and establishing relationships between tables, where a central task is to establish join relationships for a given set of input tables[1]. This closely relates to foreign-key (FK) detection [18, 30, 57] but works specifically in the context of BI, where the resulting schema graphs from the modeling step frequently correspond to structures known as star-schema and snowflake-schema studied in data warehouses [16], like shown in Figure 1.

While self-service BI tools also attempt to improve the usability of the BI-modeling step through better GUI (e.g., allowing users to specify join columns using drag-and-drop) [10, 12], building BI models remains a key pain point. This is because when faced with a large number of tables, even experienced technical users like DBAs can find the task of identifying all possible join relationships challenging and time-consuming. For less-technical enterprise users who are not familiar with concepts like fact/dimension tables, building BI models from scratch can be a daunting challenge.

Foreign-key detection: not yet sufficient. It would clearly be useful, if the join relationships in BI models can be automatically predicted on given input tables (without requiring users to specify them manually). Since joins in BI models are often primary-key (PK) foreign-key (FK) joins, existing FK detection algorithms [18, 30, 57] would seem to apply.

To study this systematically, we harvested over 100K real BI models built using self-service BI tools, from public sources like GitHub and search engines. For each BI model file, we programmatically extract the input tables used in the model, as well as the ground-truth join relationships specified by users, thus creating a real BI benchmark for large-scale evaluation for the first time (prior work mostly use synthetic benchmarks for evaluation instead).

Our large-scale evaluation using these real BI datasets suggests that existing FK-detection algorithms are still insufficient for the

*Work done at Microsoft.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 10 ISSN 2150-8097.
doi:10.14778/3603581.3603596

[1]We note that more advanced BI modeling can involve additional steps such as schema redesign and performance optimization, which is usually out of scope for non-technical users in self-service BI tools, and thus not considered in this work.

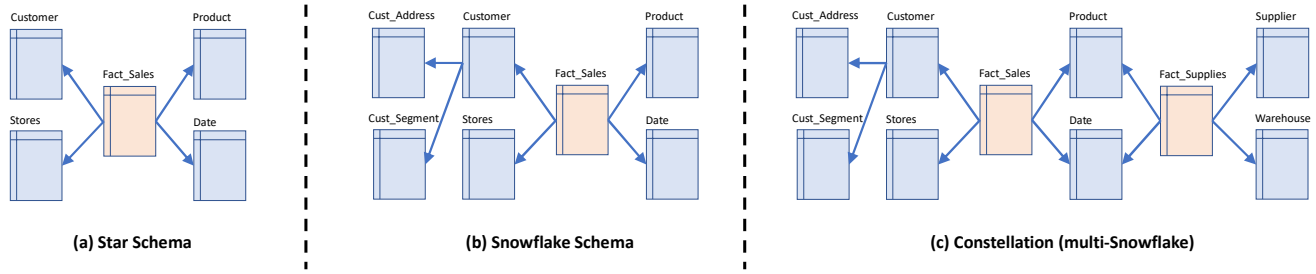


Figure 1: Types of common BI schemas. (a) Star schema, which can be seen as a simplified version of Snowflake. (b) Snowflake schema, where dimension tables like "Customers" can connect to additional dimension tables. (c) Constellation schema, which can be seen as multiple Snowflake schemas.

task, as they frequently produce incorrect results (with ~0.6 precision for all methods we tested). This is because real BI data in the wild tend to be considerably more challenging (e.g., column-headers can often be cryptic with generic names like "name" or "id", and column-values can often overlap accidentally). Crucially, most existing approaches are "local" in nature, that consider two tables at a time to make greedy local decisions, without a principled global optimization, thus unable to produce accurate predictions on challenging real BI test cases (Section 2).

Auto-BI: global optimization using schema graphs. A key insight we develop in Auto-BI, is that because we know users are finding join relationships specifically for building BI-models, and we know the typical structure that schema-graphs in BI-models should generally follow (e.g., variants of snowflake), this gives us a unique opportunity to leverage the *global graph structure* of the resulting schema-graph, to predict joins much more accurately.

We thus formulate the Auto-BI problem as a global optimization problem that holistically considers both the *local decision* of pairwise joinability, as well as the *global decision* of graph structures.

Specifically, we show that the snowflake schema popular in BI-modeling corresponds to a graph-theoretical concept called *Arborescence* [46]. Leveraging this structure, we formulate a new graph problem called *k-MCA (k-Minimum-Cost Arborescence)*, which finds the most probable k-snowflakes, by considering both local joinability and global graph structures, all using a precise probabilistic interpretation based on calibrated probabilities.

We prove that the new k-MCA problem and its variants are in general intractable (NP-hard) and inapproximable (EXP-APX-complete). Nevertheless, we develop novel algorithms based on branch-and-bound principles, which are surprisingly efficient on real test cases and can scale to the largest BI models we encounter, while still being provably optimal. Our extensive evaluations using real BI models and synthetic TPC benchmarks suggest that the proposed Auto-BI is substantially more accurate when compared to existing solutions in the literature.

Contributions. We make the following contributions:

- We are the first to harvest and leverage over 100K real BI models in the wild, for the problem of predicting BI models. This enables a data-driven algorithm design and makes it possible to rigorously evaluate different algorithms on real BI data.
- We are the first to exploit the snowflake-like schema structure in BI settings for our predictions, by mapping snowflakes to a less-known graph-theoretical concept called arboresence. We formulate a set of novel graph-based optimization problems that we call k-MCA, that have precise probabilistic foundations.
- We study the theoretical hardness of k-MCA variants, and propose efficient algorithms that are provably optimal. Extensive evaluations show that our algorithms are both effective (with over 0.9 F1 scores), and efficient (scales to the largest BI models we encounter and runs in a few seconds).

2 RELATED WORKS

BI and dashboarding tools. There are a wide variety of BI and dashboarding tools that aim to help users perform ad-hoc data analysis, with Tableau [11] and Power-BI [9] being the leading vendors [25]. These tools use visual drag-and-drop interfaces [38] (without requiring users to write SQL), and are particularly popular among non-technical users.

Foreign key detection. Foreign key (FK) detection is an important problem in database settings, with many influential methods developed in the literature [18, 30, 47, 57].

MC-FK [57] is a pioneering effort to detect FK using an EMD-based randomness metric for distribution similarity between two columns, which is more reliable to predict true relationships when unrelated key-columns that can frequently have overlapping ranges.

Fast-FK [18] develops an efficient method that selects FKs with the best pre-defined scores until all input tables are connected.

HoPF [30] improves upon prior work by considering not only FK-scores but also PK-scores, which are combined using a predefined scoring function, making this also a global method in spirit. The algorithm enumerates all PK/FK combinations and returns the combination that has the highest total score.

ML-FK [47] proposes an ML-based classifier to predict FK, trained on known FKs. As we will show analytically and experimentally, without principled global optimizations, ML classification alone is still local in nature and not sufficient to achieve high accuracy.

While FK detection methods are clearly related to our problem (we experimentally compare them with these methods), there are also a few key distinctions that we would like to highlight.

First, while FK detection targets general database settings, we focus on predicting joins in BI models, which gives us a unique opportunity to exploit the likely graph structure (e.g., snowflake) to make more accurate predictions, which is a unique direction not considered in prior work.

Second, unlike FK-detection methods that typically consider the canonical PK/FK (1:N) joins, in Auto-BI the types of joins we consider are more general, because real BI models in the wild frequently employ 1:1 joins as well as joins that are not perfectly 1:N, making the prediction problem more challenging.

Lastly, with the exception of [47], most prior FK detection methods primarily rely on hand-crafted scoring functions to make local join predictions (pairwise between two tables). In comparison, we leverage the BI models harvested to first predict local-joinability in a data-driven way, which is then combined into a principled global

optimization for optimal global decisions (at the graph-level for all tables). This also makes our technique different from prior work.

Detect inclusion dependency. Inclusion dependency (IND) is closely related to FK, with an influential body of work focusing on *efficiently* enumerating inclusion dependency (IND) in large databases [13, 33, 34, 39, 43, 49, 50, 52]. The focus on efficiency makes this line of work orthogonal to FK-detection, where the focus is to accurately predict meaningful FKs (from a large collection of IND candidates). Like prior FK-detection methods that employ efficient IND detection [30, 57], we also use IND-detection as a pre-processing step to enumerate possible FK candidates efficiently.

Complex non-equi joins. Beyond equi-joins, techniques to automatically detect and handle complex join relationships have also been studied, e.g., transformation-based join [42, 54, 59], fuzzy-join [17, 37, 51], search-join [36], semantic lookup-join [29], etc., which is an interesting area of future work in the context of BI.

3 PROBLEM STATEMENT

In this section, we first describe preliminaries and the real BI models we harvest, before introducing the Auto-BI problem.

3.1 Preliminary: BI models

Business Intelligence is closely related to topics like data warehousing and decision support system, and has been extensively studied. We provide a brief preliminary here and refer readers to surveys like [16, 32, 40] for more information.

Fact and dimension tables. In data warehousing and BI terminologies, tables involved in BI modeling can be categorized as two types: *fact tables* and *dimension tables* [32]. A *fact table* contains key metrics and measurements of business processes that one intends to analyze (e.g., the revenue of sales transactions). In addition, a fact table contains *foreign keys* that can reference multiple dimension tables, where each *dimension table* contains detailed information associated with the measurements from a unique facet (e.g., a "`Product`" dimension table contains details of products sold, whereas a "`Date`" dimension table has detailed day/month/year info of transactions, etc.). Figure 1 shows a few examples of the BI schemas, with fact and dimension tables in different colors.

Such a separation of fact/dimension tables has many benefits, such as storage/query efficiency, ease of maintenance, etc. [16, 32, 40]. The fact/dimension design is a de-facto standard in BI modeling.

Star, snowflake, and constellation schemas. In BI modeling, fact/dimension tables are frequently organized into what is known as *star/snowflake/constellation schemas* [32], like shown in Figure 1.

Star-schema refers to the cases where there is one fact table, whose foreign-key columns refer to primary-keys from one or more (non-hierarchical) dimension tables, as illustrated by Figure 1(a).

Snowflake-schema generalizes the star-schema, with dimension tables referring to each other in a hierarchical manner. For example, in Figure 1(b), the "`Customer`" dimension refers to a coarser-grained dimension "`Cust-Segment`". Similarly an "`Address`" dimension can refer to a coarser-grained "`City`", which in turn refers to "`Country`".

While there is only one fact table in star and snowflake schemas, *constellation-schema* generalizes to the cases with multiple fact tables, as shown in Figure 1(c).

We note that these three types of schemas are extensively studied in the literature [16, 32, 40] and widely adopted in practice.

3.2 Harvest Real BI Models

In order to understand real BI models created in the wild, and systematically evaluate the effectiveness of Auto-BI, we harvest over 100K real BI models from public sources that are created using a popular tool Power BI [9], whose model files have a suffix "`.pbix`".

We crawl these "`.pbix`" model files from two sources. The first is GitHub, where developers and data analysts upload their Power BI models for sharing and versioning. We crawl a total of 27K such model files from GitHub. As a second source, we go through the URLs crawled by a commercial search engine that can lead to "`.pbix`" model files. Most of these URLs are not crawled by the search engine so we crawl ourselves and obtain 86K model files.

Combining data from the two sources gives us a large collection of 100K+ real BI models, which cover diverse BI use cases, including financial reporting, inventory management, among many others.

These real BI models become a valuable dataset for rigorous evaluation – specifically, from each "`.pbix`" model file, we programmatically extract all tables used in the model, as well as the ground-truth BI model (join relationships) manually specified by users. This enables us to thoroughly evaluate different algorithms using real BI models in the wild, which turn out to be more challenging than synthetic TPC benchmarks used in prior work that tend to be clean and simple.

3.3 Problem Statement: Auto-BI

We now define the high-level Auto-BI problem as follows.

Definition 1. [Auto-BI]. Given a set of input tables $\mathbf{T} = \{T_1, T_2, \ldots, T_n\}$ used for BI modeling, where each table T_i consists of a list of columns $T_i = (c_{i1}, c_{i2}, \ldots, c_{im})$. Predict a desired BI model $M(\mathbf{T})$ that consists of a set of joins $M(\mathbf{T}) = \{J_{ij}, J_{pq}, \ldots\}$, where each join J_{ij} is a join between T_i and T_j, in the form of $J_{ij} = \big((c_{ik}, c_{il}, \ldots), (c_{jr}, c_{js}, \ldots)\big)$.

Note that the output $M(\mathbf{T})$ is restricted to equi-joins for now, which can be single-column joins, or multi-column joins in the form of $J_{ij} = \big((c_{ik}, c_{il}, \ldots), (c_{jr}, c_{js}, \ldots)\big)$. Also note that we only state the high-level problem so far, and will defer the exact graph-based formulation to Section 4.3.

A more general version of the problem goes beyond equi-joins in Definition 1 to handle complex forms of joins such as transformation-joins [59] and fuzzy-joins [37], which we will leave as future work.

4 AUTO-BI

We will start with an overview of our Auto-BI architecture, before discussing how we predict joins holistically on graphs.

4.1 Architecture Overview

Figure 2 shows the overall architecture of our Auto-BI system. It consists of an offline component and an online component, as depicted by the two boxes shown in the figure.

In the offline step, using real BI models we harvested and their ground-truth BI join models, we perform offline training to learn what we call "local join models" that can predict, for a given pair

CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning

HAOTIAN GAO, National University of Singapore, Singapore and NUSRI Chongqing, China
SHAOFENG CAI, National University of Singapore, Singapore
TIEN TUAN ANH DINH, Deakin University, Australia
ZHIYONG HUANG, National University of Singapore, Singapore and NUSRI Chongqing, China
BENG CHIN OOI, National University of Singapore, Singapore

Machine learning models are only as good as their training data. Simple models trained on well-chosen features extracted from the raw data often outperform complex models trained directly on the raw data. Data preparation pipelines, which clean and derive features from the data, are therefore important for machine learning applications. However, constructing such pipelines is a resource-intensive process that involves deep human expertise.

Our goal is to design an efficient framework for automatically finding high-quality data preparation pipelines. The main challenge is how to explore a large search space of pipeline components with the objective of computing features that maximize the performance of the downstream models. Existing solutions are limited in terms of feature quality, which results in low accuracies of the downstream models, while incurring significant runtime overhead. We present CtxPipe, a novel framework that addresses the limitations of previous works by leveraging contextual information to improve the pipeline construction process. Specifically, it uses pre-trained embedding models to capture the data semantics, which are then used to guide the selection of pipeline components. We implement CtxPipe with deep reinforcement learning and evaluate it against state-of-the-art automated pipeline construction solutions. Our comprehensive experiments demonstrate that CtxPipe outperforms all of the baselines in both model performance and runtime cost.

CCS Concepts: • **Information systems** → **Data analytics**; **Data cleaning**; • **Computing methodologies** → **Machine learning**.

Additional Key Words and Phrases: Data Analytics Pipeline, Data Preparation

ACM Reference Format:
Haotian Gao, Shaofeng Cai, Tien Tuan Anh Dinh, Zhiyong Huang, and Beng Chin Ooi. 2024. CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning. *Proc. ACM Manag. Data* 2, 6 (SIGMOD), Article 231 (December 2024), 27 pages. https://doi.org/10.1145/3698831

1 Introduction

The quality of input data determines the quality of Machine Learning (ML) models. For machine learning applications involving relational data, simple tree-based models trained with well-selected features derived from the raw data often have comparable, if not better model performance than that of state-of-the-art deep learning models trained directly on the raw data [19, 20, 40, 48]. The

Authors' Contact Information: Haotian Gao, National University of Singapore, Singapore, Singapore and NUSRI Chongqing, Chongqing, China, gaohaotian@comp.nus.edu.sg; Shaofeng Cai, National University of Singapore, Singapore, Singapore, shaofeng@comp.nus.edu.sg; Tien Tuan Anh Dinh, Deakin University, Melbourne, Victoria, Australia, anh.dinh@deakin.edu.au; Zhiyong Huang, National University of Singapore, Singapore, Singapore and NUSRI Chongqing, Chongqing, China, huangzy@comp.nus.edu.sg; Beng Chin Ooi, National University of Singapore, Singapore, Singapore, ooibc@comp.nus.edu.sg.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).
ACM 2836-6573/2024/12-ART231
https://doi.org/10.1145/3698831

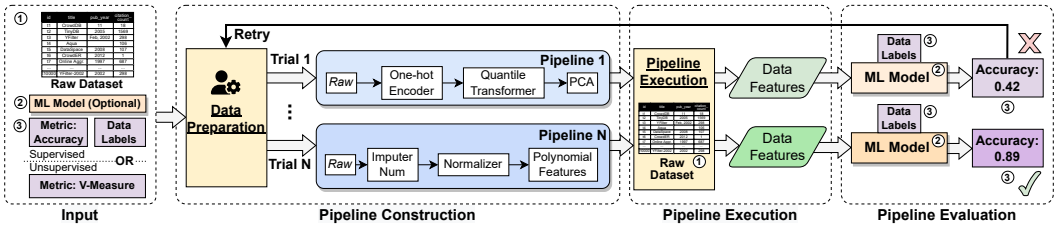


Fig. 1. Constructing a data preparation pipeline to maximize the performance of the generated data features.

tree-based models are not only more efficient to train but also have better interpretability [39, 46]. Extracting high-quality features from raw data requires a *data preparation pipeline*, which comprises multiple components that clean and transform raw data into features [1, 33]. These components either come from the ML frameworks, or are defined by the user.

The process of building a data preparation pipeline that generates high-quality features, or *data preparation pipeline construction*, is illustrated in Figure 1. It is performed by a data scientist, taking as input the raw dataset, an ML model, and its corresponding metric for evaluation. For supervised learning, the data labels are also provided. The process is manual and iterative. At each iteration, the scientist constructs a pipeline, executes it to derive a set of features, trains the model, and observes the model performance. The final output is the pipeline that achieves the best model performance.

Our goal is to design an efficient framework for automatically finding high-quality data preparation pipelines, that is, for automated data preparation pipeline construction. The main challenge is how to efficiently explore a large search space of pipeline components. In particular, the search space of possible component combinations grows exponentially with the number of possible components for different cleaning and transformation tasks [49]. Furthermore, the time cost of evaluating the pipelines is significant, because for each pipeline this cost includes the execution of the pipeline, as well as the model training and testing. With distribution shifts [32, 39, 47] requiring periodic pipeline rebuilding, this overhead accumulates and becomes significant over time.

Existing works on automated pipeline construction fall short of achieving our goal [8, 16, 21, 24, 26, 28–30, 33, 42, 46, 49, 54, 60]. Different techniques, e.g., genetic programming [42, 49], Bayesian optimization [16, 46, 54], reinforcement learning [8, 11, 21, 46], and differential formulation [24, 33, 60] are used to select and permute pipeline components. However, the search space still grows exponentially, resulting in significant runtime overhead. One reason for the large search space is that these works only use table schema and other statistical information to guide the exploration.

Our insight is that we can leverage *contextual information* to significantly reduce the search space. Intuitively, by capturing the *domain expertise* that data scientists use during the manual construction of data preparation pipelines, we can perform a more guided, efficient search. Such domain knowledge from the data, or the contextual information, can be derived from multiple sources (Figure 2). For example, the domain of the datasets (such as healthcare, finance, e-commerce, etc.) can be inferred from the column names and concrete string values in the table. The relationship across columns can be derived from columns with similar names. The high-level semantics of statistical information can also be derived from the type and range of numerical values in the columns. These column semantics, if captured automatically, could enable a more fine-grained separation of dataset scenarios to the pipeline construction solution, thereby improving its predictive performance.

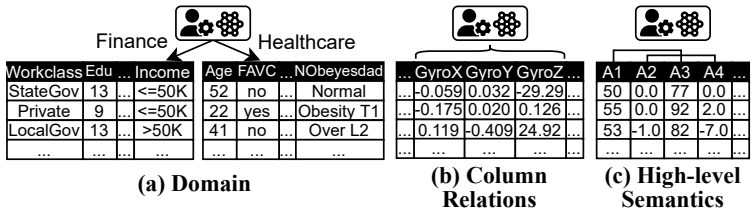


Fig. 2. Examples of contextual information.

We present CtxPipe[1], an efficient framework for automated data preparation pipeline construction. It addresses the limitations of previous works by leveraging contextual information from the data to improve the pipeline search. More specifically, CtxPipe employs state-of-the-art pre-trained embedding models to capture the semantics of raw and transformed data, then uses them to dynamically manipulate the training and inference of a pipeline construction model via a *gated context plug-in* module. CtxPipe can identify the pipeline that produces high-quality features, because pipeline search is guided within a specific context. In contrast, previous works use only table statistics to guide the search, which means that two tables with similar statistics but different semantics may produce similar pipelines and the corresponding features. CtxPipe implements the context plug-in based on deep reinforcement learning [11, 21]. It further improves the module robustness by introducing an optimized agent optimization algorithm called *Open-Closed-Gated (OCG) experience replay* that performs controlled backpropagations under marginal plug-in setups.

The most similar system of CtxPipe is HAIPipe [11], which leverages pre-built pipelines to complement the automatically generated pipelines. However, such pre-built pipelines are created manually, making the entire process less automated. In contrast, our system integrates external knowledge while enabling a fully automated pipeline construction workflow. To the best of our knowledge, we are the first to design and implement a fully automated, context-aware solution for data preparation pipeline construction.

In summary, we make the following contributions:

- We present CtxPipe, a novel and efficient framework for automated data preparation pipeline construction. It makes use of embedding models to automatically extract context into vectors, which are then integrated into the pipeline construction algorithm via a *gated context plug-in* module.
- We propose an agent optimization strategy called *Open-Closed-Gated (OCG) experience replay* that performs effective and stable joint optimization on the main deep reinforcement learning network and the context plug-in.
- We compare CtxPipe against state-of-the-art solutions in terms of the quality of transformed data on 18 commonly used datasets. Our extensive experimental results show that CtxPipe outperforms the baselines in all evaluation metrics, and achieves up to 170.99x speedup in pipeline construction compared to the second-best baseline.

The remainder of the paper is structured as follows. Section 2 reviews the related work. Section 3 presents the preliminaries and the problem definition. Section 4 overviews CtxPipe, focusing on its pipeline construction agent. Section 5 discusses the context plug-in in detail, including its context integration mechanism and agent optimization. Section 6 discusses the implementation, followed by the evaluation results in Section 7. Section 8 concludes the paper.

[1]The source code of CtxPipe is available at: https://github.com/ctxpipe.

FEATPILOT: Automatic Feature Augmentation on Tabular Data

Jiaming Liang*
University of Pennsylvania
liangjm@seas.upenn.edu

Chuan Lei, Xiao Qin, Jiani Zhang, Asterios Katsifodimos,
Christos Faloutsos, Huzefa Rangwala
Amazon Web Services
chuanlei, drxqin, zhajiani, akatsifo, faloutso, rhuzefa@amazon.com

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00163

***Abstract*—Tabular data within enterprises or open data repositories provide a huge opportunity for feature augmentation. Using these data sources to augment training data often boosts model performance, which is crucial in data-centric AutoML systems. Recent works on automatic feature augmentation have limited capabilities in utilizing useful features that cannot be joined with the base table without connecting through intermediate tables. We present FEATPILOT, a novel framework that explores and integrates high-quality features in tabular data for ML models. FEATPILOT evaluates a candidate feature from two aspects: (1) the efficacy of a join path connecting the feature to the base table and (2) the intrinsic value of a feature towards an ML task. FEATPILOT efficiently identifies high-quality features and their optimized join paths to augment the base table. Our experimental results show that FEATPILOT achieves up to a 10.27% improvement in ML model performance compared to state-of-the-art solutions across six public datasets.**

I. INTRODUCTION

Feature augmentation on tabular data is crucial for improving machine learning (ML) model performance [1]–[5]. With additional features, models can capture more complex patterns and relationships within the data, leading to better prediction and generalization results. By exploring and integrating features [6], [7] from open tabular data repositories (e.g., Google Dataset Search[1], Kaggle[2], and OpenML[3]), ML models [8] can avoid overfitting [9] and improve performance on unseen data [3]–[5].

Given a base table with an ML task defined on it and a set of candidate tables that can be joined with the base table via one-hop or multi-hop paths, feature augmentation is the task of selecting feature columns from the candidate tables that can maximize the performance of the ML task. This process typically involves identifying useful features that can reveal additional insights or patterns for the given ML task, and determining the optimal paths to join these candidate features with the base table to enrich the initial training data.

Motivating Example. An ML engineer is developing a classification model to predict whether a product should be recommended or not. The base table `product` contains only a limited set of features for building a high-quality model (shown in Fig. 1). Fortunately, useful features exist in related tables such as `address` in the `customer` table. To augment the base table, the ML engineer needs to find a high-quality join path to augment the base table. Among three join paths in Fig. 1, `customer` ⋈ `order` ⋈ `product` (indicated by the thick red line) provides more feature instances (i.e., `address`) compared to paths involving the `rating` or `review` table, as customers may not provide ratings or reviews for every purchased product. As a result, the ML engineer can exploit this augmented table to improve the classification model.

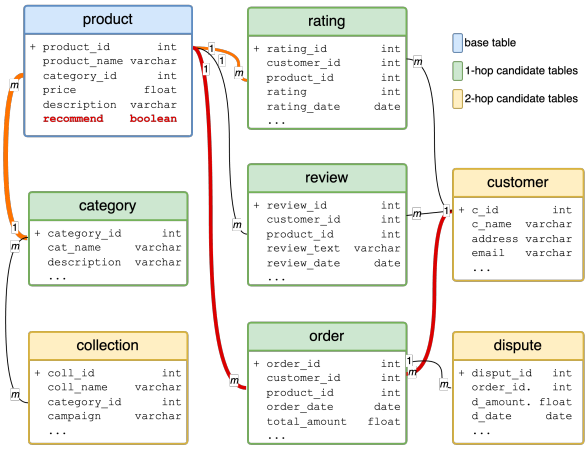


Fig. 1: Example of feature augmentation. The base table, `product`, is in blue. The task is to predict the values in the Boolean type column 'recommend'. The tables in green and yellow are 1-hop and 2-hop joinable with the base table.

Challenges. In this paper, we address the following three key challenges in feature augmentation.

(1) *Multiple Join Paths Selection.* In real-world open data settings, a candidate feature can be integrated with the base table via multiple join paths, resulting in different integration quality as explained in the motivating example. Hence, it is critical to identify the optimal join path to achieve the full potential of a candidate feature.

(2) *Expensive Join Operations.* To find out the actual benefit of augmenting the base table with a candidate feature, expensive joins need to be executed, which is computationally expensive and time consuming, especially when a long join path involves multiple joins. Candidate features in open data repositories often have similar join paths. Therefore, sharing join operations is crucial for efficient feature augmentation.

*The work was done when Jiaming Ling was with Amazon Web Services.
[1]https://datasetsearch.research.google.com/
[2]https://www.kaggle.com/datasets
[3]https://www.openml.org/

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00163

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

(3) *Search Complexity.* The search space of all candidate tables and their join paths is vast and intractable, making it prohibitively expensive to find the optimal set of candidate tables. To address this challenge, we need effective strategies to reduce the search space while preserving optimality.

State-of-the-art Approaches. Prior approaches [3], [10] augment the base table by joining other tables using either their relevancy to the base table or the improvement of the candidate features bring to the ML models. Recently, ALITE [11] and DIALITE [12] are introduced to integrate tables based on joinability or unionability, which can improve the quality of downstream ML models. However, these approaches assume that the features and their respective relevance to the base table are specified. Hence, they fail to leverage all possible candidate features and consequently do not achieve optimal model performance. Reinforcement learning (RL) [13], [14] has emerged as an effective approach to balance between feature exploration and exploitation. Several studies [4], [5], [15] leverage RL techniques such as multi-armed bandits and Deep Q-Networks (DQNs) to explore the large search space of candidate tables. Although these RL-based methods have achieved promising results, they still face critical challenges. Specifically, METAM [5] is limited to directly joinable candidate tables and AutoFeature [4] does not provide fine-grained optimization on integration quality. Moreover, RL often requires extensive training data and tends to overfit, reducing its effectiveness in real-world applications.

Proposed Solution. To cope with the above mentioned challenges, we propose FEATPILOT, an efficient and effective framework for automatic feature augmentation. Given a base table, a set of candidate tables and an ML task, FEATPILOT first selects a subset of candidate tables using a clustering algorithm based on table characteristics. FEATPILOT further chooses representative join paths from these candidate tables to the base table. This allows FEATPILOT to discover the actual performance gain led by the candidate tables following the join paths (*Challenge 1*). Consequently, FEATPILOT learns an integration quality model and a feature importance estimator to calculate the performance gain of any unexplored candidate tables and their respective join paths (*Challenge 2*). Lastly, FEATPILOT's join-path search algorithm prunes the search space based on both integration quality and feature importance, and finds the optimized integration paths for the selected candidate tables in polynomial time (*Challenge 3*).

Contributions. Our main contributions are the following.

(1) We introduce FEATPILOT, a novel feature augmentation framework for ML tasks over tabular data repositories, by decomposing the intractable problem of feature augmentation into feature importance and integration quality.

(2) We design an effective feature exploration module consisting of a clustering-based feature importance estimation and a learning-based integration quality model.

(3) We propose an efficient feature path search algorithm that exploits both estimated feature importance and integration quality to identify high-quality features and their optimized join paths for integration.

(4) Our experiments show that FEATPILOT achieves up to a 10.27% improvement in ML model performance compared to state-of-the-art solutions across six public datasets.

Reproducibility. The source code of FEATPILOT is available at `https://anonymous.4open.science/r/FeaturePilot-C308`.

II. Preliminaries and System Overview

A. Basic Notions

Definition 1 (Base Table): The base table, T_{base} = $\{c_1, c_2, \ldots, c_n, c_l\}$, refers to the original table that includes the initial set of features and the label column before any augmentation. Each c_i ($i \in [1, n]$) represents a column and c_l denotes the label column, which stores the prediction target for each row.

Definition 2 (Candidate Tables): The candidate tables, $\mathcal{T}$ = $\{T_1, T_2, \ldots, T_m\}$, are m joinable tables that can be used to augment a base table T_{base}. Each table is denoted by T_j = $\{f_{j,1}, f_{j,2}, \ldots, f_{j,k}\}$ ($j \in [1, m]$), where $f_{j,k}$ denotes the k-th column (a.k.a feature)[4] in table T_j.

Definition 3 (Join Path): Given a set of tables $T_1, T_2, \ldots, T_n$, a join path P can be defined as a sequence of join operations between pairs of tables. A join operation between any two tables T_i and T_j can be denoted as $T_i \bowtie T_j$. The join path P can be represented as: $P = T_1 \bowtie T_2 \bowtie T_3 \ldots \bowtie T_n$.

Without loss of generality, we allow different types of join operators between T_{base} and $T_j \in \mathcal{T}$, including inner join, left outer join, right outer join, etc.

Definition 4 (Augmented Table): The augmented table $T_{aug} = T_{base} \bowtie_P T_j$ is the result of joining the base table T_{base} and a candidate table T_j following a join path P.

Note that there can be multiple join paths between the base table and a candidate table. Hence the augmented table could be different depending on the chosen join path.

Definition 5 (Machine Learning Task): A machine learning (ML) task is defined as a function f that maps an input space $\mathcal{X}$ to an output space $\mathcal{Y}$. Given a dataset $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$, where $x_i \in \mathcal{X}$ and $y_i \in \mathcal{Y}$, the objective is to learn the function f that best approximates the true underlying relationship from the inputs to the outputs.

Definition 6 (Utility Gain): Given a ML task, the utility score, $US(T)$, is defined as the performance metric (e.g., accuracy, precision, recall, etc.) of the task on a table T achieved by a specific ML model[5]. Let $US(T_{base})$ be the performance metric on the base table, and $US(T_{aug}) = US(T_{base} \bowtie_P T_j)$ be the performance metric on the augmented table. The utility gain UG is defined as the improvement in the performance metric: $UG = US(T_{aug}) - US(T_{base})$.

Problem Statement. Given a base table T_{base}, a set of candidate tables $\mathcal{T}$, and a ML task, our goal is to select features from these candidate tables $\mathcal{T}$ and to identify the best join paths to augment the base table T_{base} with the selected

[4]We use two terms, column and feature, alternately in this paper.

[5]$US(T)$ uses the negation of a metric value when smaller values are preferred (e.g., mean squared error (MSE), mean absolute error (MAE), etc.).

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

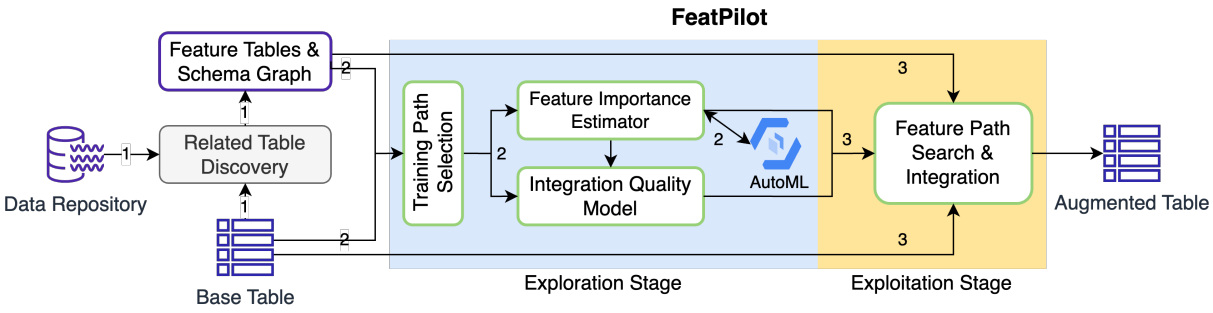


Fig. 2: FEATPILOT overview.

features, such that the utility gain UG of the ML task on T_{aug} is maximized. Formally,

$$\max_{\mathcal{S}\subseteq\mathcal{T},\mathcal{P}} UG = US(T_{aug}) - US(T_{base}) \quad \text{subject to } T_{aug} = T_{base} \bowtie_{\mathcal{P}} \mathcal{S} \tag{1}$$

where $\mathcal{S} \subseteq \mathcal{T}$ is the subset of selected candidate tables from $\mathcal{T}$, $\mathcal{P}$ is the set of join paths used to augment T_{base} with $\mathcal{S}$, and T_{aug} is the augmented table obtained by joining T_{base} with the selected tables $\mathcal{S}$ following the join paths $\mathcal{P}$.

Note that we treat the ML model (e.g., linear regression, deep neural network, XGBoost etc.) and the corresponding task as a black box such that the user can leverage our framework to augment features regardless of any particular model, and the model parameters can be updated iteratively.

The complexity of feature augmentation problem generally increases exponentially with the number of candidate features and all possible join paths among the base and candidate tables. The combination of these two exponential search spaces makes the problem NP-hard. Even with various heuristic and optimization techniques [16], [17] to reduce the complexity, the cost of feature augmentation can be still expensive.

B. FEATPILOT System Overview

For a given ML task, FEATPILOT takes input as a base table with a prediction task on the target column and a tabular data repository consisting of candidate tables with features. As depicted in Fig. 2, FEATPILOT leverages data discovery systems such as Aurum [18] or NYU Auctus [19] to identify the candidate tables within k-hop joins to the base tables. FEATPILOT then triggers its utility gain model in two stages, i.e., exploration and exploitation.

FEATPILOT's exploration stage tackles *Challenge 1* and *Challenge 2* mentioned in Section I. It takes the following four steps. (1) FEATPILOT groups the identified features into clusters based on their representations (embeddings, data distributions, data types, etc.). (2) FEATPILOT employs a path selection strategy to choose a set of features and candidate join paths. (3) FEATPILOT then follows the selected join paths to integrate the candidate features into the base table and runs AutoML (e.g., AutoGluon[6]) to compute the utility gain over the model trained on the base table. (4) Using the data collected in (3), FEATPILOT trains an integration quality model and assigns the actual feature importance to the selected features in the clusters. The importance of the other features can be estimated based on their relative distance to the ones with the actual importance assigned.

FEATPILOT's exploitation stage address *Challenge 3* discussed in Section I. It leverages both the feature importance estimator and integration quality model to conduct an efficient and effective search over all candidate features and their associated join paths. It iteratively selects the most important feature and uses the integration quality model to select the join path with the best integration quality until a predefined number of features is reached. FEATPILOT then follows the selected join paths to integrate the selected features with the base table and generates the final augmented table for the given ML task. Finally, AutoML solutions can train, select and deploy the best performing ML model on the augmented table [20].

III. UTILITY GAIN MODELING

The problem of *maximizing utility gain* for a given ML task is intractable. Hence we propose to decompose this problem into two sub-problems, namely maximizing feature importance – how useful is a feature to the ML model, and maximizing integration quality – how many data instances in the base table can be augmented by a feature.

Feature Importance. To define the *feature importance*, we first introduce the concept of a *virtual table*. Let T_{aug} denote a resulting augmented table from a base table T_{base} and a candidate table T_c. The virtual table T_{virt} refers to the ideal case where there exists a join path P such that every instance in T_{base} can be augmented by a feature instance from T_c via this P (i.e., iPhone 15 with a rating of 5, Pixel 8 with a rating of 4.5, and Galaxy S23 with a rating of 4 shown in Fig. 3b). Note that a virtual table is specific to a base table and an individual candidate feature.

Note that, for ease of illustration, we assume a 1:1 join relationship in this example. In practice, FEATPILOT supports 1:1, 1:m, and m:m join relationships between the base table T_{base} and a candidate table T_j. To handle these scenarios, FEATPILOT employs commonly used methods based on feature data types, including aggregation (e.g., count, mean, min/max, etc. for numerical features), pooling on feature embeddings (for textual features), one-hot encoding (for categorical features).

[6] https://auto.gluon.ai/

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:44:54 UTC from IEEE Xplore. Restrictions apply.

HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation

SIBEI CHEN, Renmin University of China, China
NAN TANG, QCRI / HKUST (GZ), Qatar / China
JU FAN*, Renmin University of China, China
XUEMI YAN, Renmin University of China, China
CHENGLIANG CHAI, Beijing Institute of Technology, China
GUOLIANG LI, Tsinghua University, China
XIAOYONG DU, Renmin University of China, China

Data preparation is crucial in achieving optimized results for machine learning (ML). However, having a good data preparation pipeline is highly non-trivial for ML practitioners, which is not only domain-specific, but also dataset-specific. There are two common practices. Human-generated pipelines (**HI-pipelines**) typically use a wide range of *any* operations or libraries but are highly experience- and heuristic-based. In contrast, machine-generated pipelines (**AI-pipelines**), *a.k.a.* AutoML, often adopt a predefined set of sophisticated operations and are search-based and optimized. These two common practices are mutually complementary. In this paper, we study a new problem that, given an HI-pipeline and an AI-pipeline for the same ML task, can we combine them to get a new pipeline (**HAI-pipeline**) that is better than the provided HI-pipeline and AI-pipeline? We propose HAIPipe, a framework to address the problem, which adopts an enumeration-sampling strategy to carefully select the best performing combined pipeline. We also introduce a reinforcement learning (RL) based approach to search an optimized AI-pipeline. Extensive experiments using 1400+ real-world HI-pipelines (Jupyter notebooks from Kaggle) verify that HAIPipe can significantly outperform the approaches using either HI-pipelines or AI-pipelines alone.

CCS Concepts: • **Information systems → Data analytics**.

Additional Key Words and Phrases: pipeline generation, data preparation, reinforcement learning

ACM Reference Format:
Sibei Chen, Nan Tang, Ju Fan, Xuemi Yan, Chengliang Chai, Guoliang Li, and Xiaoyong Du. 2023. HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation. *Proc. ACM Manag. Data* 1, 1, Article 91 (May 2023), 26 pages. https://doi.org/10.1145/3588945

1 INTRODUCTION

Data preparation (data prep for short) is the process of turning raw data into a format that is ready for use by data science or ML tasks. This process is often implemented by a series of steps (*a.k.a.* a

*Ju Fan is the corresponding author.

Authors' addresses: Sibei Chen, Renmin University of China, Beijing, China, sibei@ruc.edu.cn; Nan Tang, QCRI / HKUST (GZ), Doha / Guangzhou, Qatar / China, ntang@hbku.edu.qa; Ju Fan, Renmin University of China, Beijing, China, fanj@ruc.edu.cn; Xuemi Yan, Renmin University of China, Beijing, China, yxmlpzt@ruc.edu.cn; Chengliang Chai, Beijing Institute of Technology, Beijing, China, ccl@bit.edu.cn; Guoliang Li, Tsinghua University, Beijing, China, liguoliang@tsinghua.edu.cn; Xiaoyong Du, Renmin University of China, Beijing, China, duyong@ruc.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART91 $15.00
https://doi.org/10.1145/3588945

pipeline) that the data moves through from one step to its subsequent steps. However, data prep is also known to be very hard and time-consuming [26].

Traditional data prep pipelines are human orchestrated, which are abbreviated as HI-pipelines. The pros of HI-pipelines are that users can use "any" operation and can easily inject domain knowledge (*e.g.,* "?" means a missing value for occupation). The cons are that users typically orchestrate pipelines based on their experience, which are heuristic and are rarely optimized, *e.g.,* humans never manually try all different combinations and find the best one.

Recent advances in the ML community have extended AutoML from hyper-parameter tuning and neural architecture search to an attempt that tries to automate the process of data prep for ML (*e.g.,* Auto-SKLearn [9] and DeepLine [12]) – these machine-generated pipelines are called AI-pipelines in this paper. The pros of AI-pipelines are that they use a constrained set of operations, for which they can find the best pipeline through exhaustively evaluating different pipelines within a pre-defined search space. The cons of AI-pipelines are that they lack domain knowledge.

Our **key observation** is that HI-pipelines and AI-pipelines are naturally complementary. That is, the pros of HI-pipelines (*i.e.,* using a wider range of operations based on domain knowledge) are the cons of AI-pipelines, and the cons of HI-pipelines (*i.e.,* experience-based and heuristic-based) are the pros of AI-pipelines (*i.e.,* highly optimized). Therefore, intuitively, HI-pipelines and AI-pipelines should be combined, so as to get the best of both worlds.

In this paper, we study a new problem: Given an HI-pipeline and an AI-pipeline for an ML data prep task, can we generate a better data prep pipeline (*i.e.,* an HAI-pipeline) by combining them?

Example 1. *Consider Figure 1. After loading the dataset (lines 1–4), an HI-pipeline consists of two components h_1 (lines 5-8,* i.e., *dropping rows with missing values) and h_2 (lines 24-28,* i.e., *scaling numerical attributes). Then, we use a machine-generated AI-pipeline that consists of three components a_1 (lines 10–15,* i.e., *encoding categorical attributes), a_2 (lines 17–22,* i.e., *generating interaction features) and a_3 (lines 30–35,* i.e., *removing useless features). A combination of these two pipelines is given in Figure 1, which outperforms both the HI-pipeline and the AI-pipeline.*

To address the problem, we propose HAIPipe, a framework that combines an HI-pipeline with an AI-pipeline for a specific ML task.

There are **two challenges**. (1) How to combine an HI-pipeline and an AI-pipeline? (2) How to obtain an optimized AI-pipeline from various possible combinations?

For Challenge (1), we devise an *enumeration-sampling* approach that first enumerates possible HAI-pipelines, and then employs an active learning strategy to sample a limited number of HAI-pipelines for evaluation, so as to find the one with the highest performance. For Challenge (2), there are a variety of operation families (*e.g.,* `Imputer` and `Scaler`). Within each family, there are many operations (*e.g.,* `MinMaxScaler` and `StandardScaler`). Thus, there exist many possible ways of selecting operations to form an AI-pipeline, which are highly dataset-specific and task-specific. We introduce a reinforcement learning (RL) based approach that learns how to select operations through offline training.

Contributions. Our contributions are summarized as follows.

(1) We formally define data prep pipelines, and introduce HI-pipelines and AI-pipelines (Section 2).

(2) We propose a framework, namely HAIPipe, that combines HI-pipelines and AI-pipelines for data prep (Section 3). We devise an enumeration-sampling strategy that effectively and efficiently combines HI-pipelines and AI-pipelines (Section 4).

(3) We present an RL-based AI-pipeline algorithm that pays particular attention on optimizing data prep operations (Section 5).

(4) We conduct extensive experiments to show that HAIPipe can significantly outperform both the

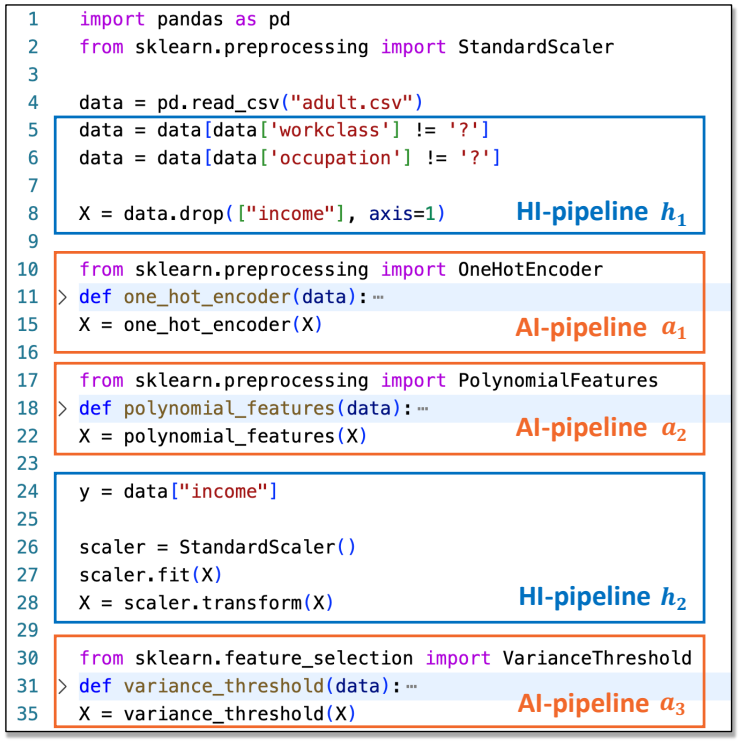


Fig. 1. An example of combining an HI-pipeline and an AI-pipeline. Blue blocks: human-written scripts. Orange blocks: machine-generated scripts.

HI-pipelines and AI-pipelines (Section 6). For reproducibility, we make code and datasets in our experiments public on GitHub[1].

2 DATA PREP PIPELINES

Let $\mathcal{T}(D, M)$ be an ML task with an ML model M and training data D. Without loss of generality, this paper considers classification models for ease of presentation. Our techniques can be easily extended to other supervised ML models. Practically, finding the "best" model M often requires a sequence of *data prep steps* to obtain the "best" data, including imputation, scaling, etc., which are naturally formalized as *operations* as below.

Data prep operations. A *data prep operation* (or operation for short), which is denoted as o, encapsulates a specific functionality that transforms a dataset D into another dataset D', *i.e.,* $D' = o(D)$. Let Σ denote the universe of operations for our task $\mathcal{T}$, which contains a set of diversified *families* of data prep functionalities. Among them, some rely on well-developed algorithms in known Python libraries, such as Scikit-Learn [27]. In this paper, we consider the following families of data prep algorithms implemented in Scikit-Learn (*i.e.,* `sklearn`), likewise in some existing studies [12, 35].

- `Imputer`: a data cleaning step that fills in missing values in dataset D, *e.g.,* `impute.SimpleImputer`.

[1]https://github.com/ruc-datalab/Haipipe



Auto-Tables: Relationalize Tables without Using Examples

Peng Li*
Georgia Tech
Atlanta, GA
pengli@gatech.edu

Yeye He
Microsoft Research
Redmond, WA
yeyehe@microsoft.com

Cong Yan
Microsoft Research
Redmond, WA
coyan@microsoft.com

Yue Wang
Microsoft Research
Redmond, WA
wanyue@microsoft.com

Surajit Chaudhuri
Microsoft Research
Redmond, WA
surajitc@microsoft.com

ABSTRACT

Relational tables, where each row corresponds to an entity and each column corresponds to an attribute, have been the standard for tables in relational databases. However, such a standard cannot be taken for granted when dealing with tables "in the wild". Our survey of real spreadsheet-tables and web-tables shows that over 30% of such tables do not conform to the relational standard, for which complex table-restructuring transformations are needed before these tables can be queried easily using SQL-based tools. Unfortunately, the required transformations are non-trivial to program, which has become a substantial pain point for technical and non-technical users alike, as evidenced by large numbers of forum questions in places like StackOverflow and Excel/Tableau forums.

We develop an Auto-Tables system that can automatically synthesize pipelines with multi-step transformations (in Python or other languages), to transform non-relational tables into standard relational forms for downstream analytics, obviating the need for users to manually program transformations. We compile an extensive benchmark for this new task, with 244 real test cases collected from user spreadsheets and online forums. Our evaluation suggests that Auto-Tables can successfully synthesize transformations for over 70% of test cases at interactive speeds, without requiring any input from users, making this an effective tool for users to prepare data for downstream analytics.

1. INTRODUCTION

Modern data analytics like SQL and BI are predicated on a standard format of relational tables, where each row corresponds to a distinct "entity", and each column corresponds to an "attribute" for the entities that contains homogeneous data-values. While such tables are de-facto standards in relational databases, to the extent that we as database people may take them for granted, we would like the highlight that a significant fraction of tables "in the wild" actually fail to conform to such standards, making it considerably more challenging to query them using SQL-based tools.

Non-relational tables are common, but hard to query. Real tables in the wild, such as spreadsheet-tables or web-tables, can often be "non-relational" and hard to query, unlike tables that we expect to find in relational databases. We randomly sampled hundreds of user spreadsheets (in Excel), and web tables (from Wikipedia), and found around 30-50% tables to have such issues. Figure 1 and Figure 2 show real samples taken from spreadsheets and the web, respectively, to demonstrate these common issues. (We emphasize that the problem is prevalent at a very large scale, since there are millions of tables like these in spreadsheets and on the web.)

Take Figure 1(a) for example. The table on the left is not a standard relational table, because each column marked in green contains sales numbers for only a single day ("`19-Oct`", "`20-Oct`", etc.), making these column values highly homogeneous in the horizontal direction (while in typical relational tables, we expect values in columns to be homogeneous in the vertical direction). Although this specific table format makes it easy for humans to eyeball changes day-over-day by reading horizontally, it is unfortunately hard to analyze using SQL. Imagine that one needs to compute the 14-day average of sales, starting from "`20-Oct`" – for this table, one has to write: `SELECT SUM("20-Oct", "21-Oct", "22-Oct", ...) FROM T`, across 14 different columns, which is long and unwieldy to write. Now imagine we need 14-day moving averages with every day in October as the starting date – the resulting SQL is highly repetitive and hard to manage.

In contrast, consider a transformed version of this table, shown on the right of Figure 1(a). Here the homogeneous columns in the original table (marked in green) are transformed into only two new columns: "`Date`" and "`Units Sold`", using a transformation operator called "`stack`" (listed in the first row of Table 1). This transformed table contains the same information as the original table, but is much easier to query – e.g., the same 14-day moving average can be computed using a succinct range-predicate on the '`Date`" column, where the starting date "`20-Oct`" is a literal parameter that can be easily changed into other values.

There are many such spreadsheet tables that require different kinds of transformations before they are ready for SQL-based analysis. Figure 1(b) shows another example, where every 3 columns form a repeating group, representing "`Revenue/Units Sold/Margin`" for a different year (marked in red/green/blue in the figure). Tables with these repeat-

*Part of work done while at Microsoft.

Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. This is a minor revision of the paper entitled "Auto-Tables: Synthesizing Multi-Step Transformations to Relationalize Tables without Using Examples", published in PVLDB, Vol. 16, No. 11, 2150-8097. DOI: https://doi.org/10.14778/3611479.3611534

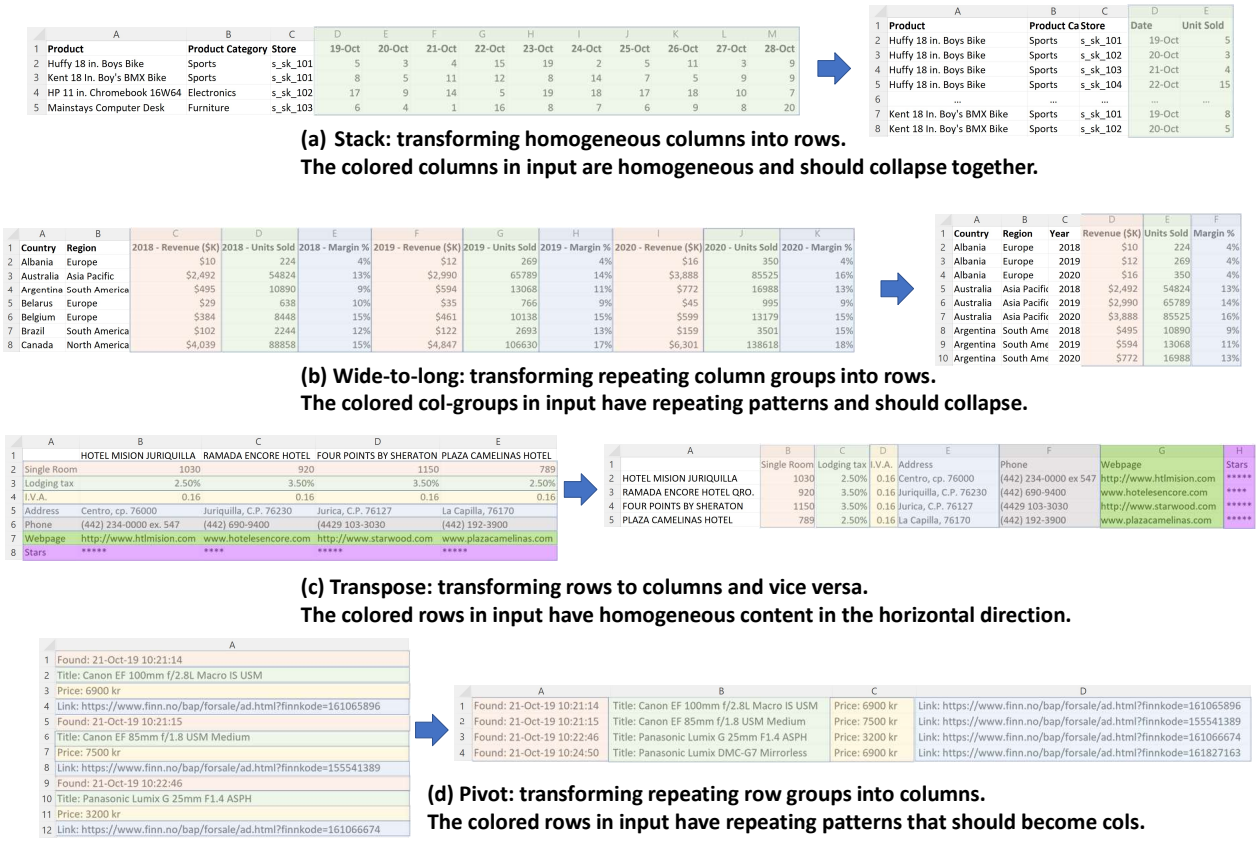


Figure 1: Example input/output tables for 4 operators in AUTO-TABLES: (a) Stack, (b) Wide-to-long, (c) Transpose, (d) Pivot. The input-tables (on the left) are not relational and hard to query, which need to be transformed to produce corresponding output-tables (on the right) that are relational and easy to query. Observe that the color-coded, repeating row/column-groups are "visual" in nature, motivating a CNN-like architecture like used in computer vision for object-detection.

ing column-groups are also hard to query, just like Figure 1(a), but in this case the required transformation operator is called "`wide-to-long`" (second row of Table 1).

Figure 1(c) shows yet another example, where each hotel corresponds to a column (whose names are in row-1), and each "attribute" of these hotels corresponds to a row. Note that in this case values in the same rows are homogeneous (marked in different colors), unlike relational tables where values in the same columns are homogeneous. A transformation called "`transpose`" is required in this case (listed in the third row of Table 1), to make the resulting table, shown on the right of the figure, easy to query – for instance, a query to sum up the total number of hotel rooms is hard to write on the original table, but can be easily achieved using a simple SUM query on the "`Single Room`" column in the transformed table.

Figure 1(d) shows another example where columns are represented as rows in the table on the left. This is similar to Figure 1(c), except that the rows in this case are "repeating" in groups, thus requiring a different transformation operator called "`pivot`" (listed in the fourth row of Table 1) as opposed to "`transpose`". The resulting table is shown on the right, which becomes easy to query.

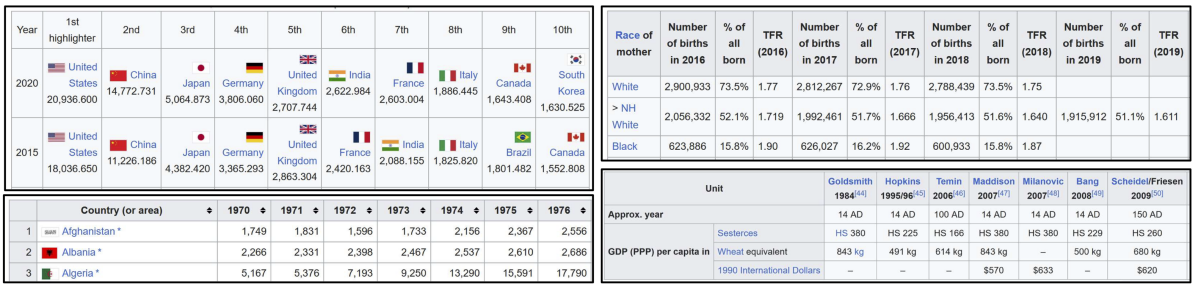
While the examples so far are all taken from spreadsheets, we note that similar structural issues are also widespread in Web tables. Figure 2 shows real examples from Wikipedia, which share similar characteristics as the spreadsheet tables in Figure 1, which all require transformations before these tables can be queried effectively.

Non-relational tables are hard to "relationalize". We mentioned that the example tables in Figure 1 and Figure 2 require different transformation operators. Table 1 shows 8 such transformation operators commonly needed to relationalize tables (where the first 4 operators correspond to the examples we see in Figure 1).

The first column of Table 1 shows the name of the "logical operator", which may be instantiated differently in different languages (e.g., in Python or R), with different names and syntax. The second column of the table shows the equivalent Pandas operator in Python [11], which is a popular API for manipulating tables among developers and data scientists, that readers may be familiar with.

While the operations listed in Table 1 already exist in languages such as R and Python, they are not easy for users to invoke correctly, because users need to:

1. Visually identify different structural issues in an input table that make it hard to query (e.g., repeating row-/column-groups shown in Fig. 1(a-d)), which is not obvious to non-expert users;
2. Map the visual pattern identified from the input table, to a corresponding operator in Table 1 that can handle such issues. This is hard for users not familiar with the exact terminologies to describe these transformations (e.g., `pivot` vs. `stack`);



Year	1st highlighter	2nd	3rd	4th	5th	6th	7th	8th	9th	10th
2020	United States 20,936.600	China 14,772.731	Japan 5,064.873	Germany 3,806.060	United Kingdom 2,707.744	India 2,622.984	France 2,603.004	Italy 1,886.445	Canada 1,643.408	South Korea 1,630.525
2015	United States 18,036.650	China 11,226.186	Japan 4,382.420	Germany 3,365.293	United Kingdom 2,863.304	France 2,420.163	India 2,088.155	Italy 1,825.820	Brazil 1,801.482	Canada 1,552.808

Race of mother	Number of births in 2016	% of all born	TFR (2016)	Number of births in 2017	% of all born	TFR (2017)	Number of births in 2018	% of all born	TFR (2018)	Number of births in 2019	% of all born	TFR (2019)
White	2,900,933	73.5%	1.77	2,812,267	72.9%	1.76	2,788,439	73.5%	1.75			
> NH White	2,056,332	52.1%	1.719	1,992,461	51.7%	1.666	1,956,413	51.6%	1.640	1,915,912	51.1%	1.611
Black	623,886	15.8%	1.90	626,027	16.2%	1.92	600,933	15.8%	1.87			

	Country (or area)	1970	1971	1972	1973	1974	1975	1976
1	Afghanistan *	1,749	1,831	1,596	1,733	2,156	2,367	2,556
2	Albania *	2,266	2,331	2,398	2,467	2,537	2,610	2,686
3	Algeria *	5,167	5,376	7,193	9,250	13,290	15,591	17,790

Unit		Goldsmith 1984[44]	Hopkins 1995/96[45]	Temin 2006[46]	Maddison 2007[47]	Milanovic 2007[48]	Bang 2008[49]	Scheidel/Friesen 2009[50]
Approx. year		14 AD	14 AD	100 AD	14 AD	14 AD	14 AD	150 AD
GDP (PPP) per capita in	Sesterces	HS 380	HS 225	HS 166	HS 380	HS 380	HS 229	HS 260
	Wheat equivalent	843 kg	491 kg	614 kg	843 kg	–	500 kg	680 kg
	1990 International Dollars	–	–	–	$570	$633	–	$620

Figure 2: Real Web tables from Wikipedia that are also non-relational, similar to the spreadsheet tables shown in Figure 1.

Table 1: Auto-Tables DSL: table-restructuring operators and their parameters to "relationalize" tables. These operators are common and exist in many different languages, like Python Pandas and R, sometimes under different names.

DSL operator	Python Pandas equivalent	Operator parameters	Description (example in parenthesis)
stack	melt [14]	start_idx, end_idx	collapse homogeneous cols into rows (Fig. 1a)
wide-to-long	wide_to_long [18]	start_idx, end_idx, delim	collapse repeating col-groups into rows (Fig. 1b)
transpose	transpose [17]	-	convert rows to columns and vice versa (Fig. 1c)
pivot	pivot [15]	repeat_frequency	pivot repeating row-groups into cols (Fig. 1d)
explode	explode [12]	column_idx, delim	convert composite cells into atomic values
ffill	ffill [13]	start_idx, end_idx	fill structurally empty cells in tables
subtitles	copy, ffill, del	column_idx, row_filter	convert table subtitles into a column
none	-	-	no-op, the input table is already relational

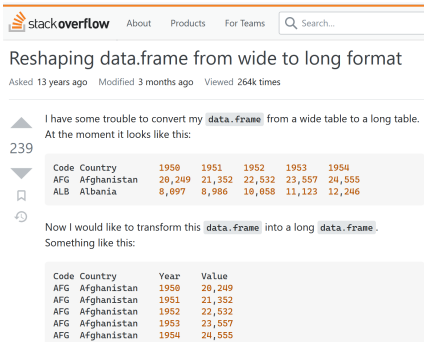


Figure 3: Example user question from StackOverflow, on how to restructure tables. Questions like this are common not only among technical users, but also non-technical users, as similar questions are commonly found on forums for Excel, Power-BI, and Tableau users too [6, 7, 8, 9].

3. Parameterize the chosen operator appropriately (e.g., which columns to collapse, what is the repeating frequency, etc.). This is again hard, as even developers need to consult API documentations that can be long and complex.
4. Certain input tables require more than one transformation step, for which users need to repeat steps (1)-(3).

Completing these steps is a tall order even for technical users, as evidenced by a large number of such questions on forums like StackOverflow. Figure 3 shows one example (popular with many up-votes) – the developer provides example input/output tables to demonstrate the desired transformation, in order to ask what operators should be used.

If technical users like developers find it hard to restructure their tables as these StackOverflow questions would show, it comes as no surprise that non-technical enterprise users, who often deal with tables in spreadsheets, would find the task even more challenging. We find a large number of similar questions on Excel and Tableau forums (e.g., [6, 7, 8, 9]), where users complain that without the required transformations it is hard to analyze data using SQL-based or Excel-based tools (e.g., [2, 3, 4, 5]). The prevalence of these questions confirms table-restructuring as a common pain point for both technical and non-technical users.

Auto-Tables: synthesize transformations without examples. In this work, we propose a new paradigm to automatically synthesize table-restructuring steps to relationalize tables, using the operators in Table 1, *without requiring users to provide examples*. Our key intuition of why we can do away with examples in our task, lies in the observation that given an input table, the logical steps required to relationalize it are *almost always unique*, as the examples in Figure 1 would all show. This is because the transformations required in our task only "restructure" tables, that do not actually "alter" the table content, which is unlike prior work that focuses on *row-to-row transformations* (e.g., TDE [31] and FlashFill [29]), or SQL-by-example (e.g. [22, 50]), where the output is "altered" that can produce many possible outcomes, which would require users to provide input/output examples to demonstrate the desired outcome.

For our task, we believe it is actually important *not* to ask users to provide examples, because in the context of table-to-table transformations like in our case, asking users to provide examples would mean users have to specify *an output table*, which is a substantial amount of typing effort, making it cumbersome to use.

As humans, we can "visually" recognize rows/columns patterns (e.g., homogeneous value groups, as color-coded vertically and horizontally in Figure 1), to correctly predict which operator to use. The question we ask in this paper, is whether an algorithm can "learn" to recognize such patterns by scanning the input tables alone, to predict suitable transformations, in a manner that is analogous to how computer-vision algorithms would scan a picture to identify common but more complex objects like dogs and cats.

In computer vision, in order to pick up subtle clues from

CM-Explorer: Dissecting Data Ingestion Problems

Niels Bylois
UHasselt
Data Science Institute, ACSL
Belgium
niels.bylois@uhasselt.be

Frank Neven
UHasselt
Data Science Institute, ACSL
Belgium
frank.neven@uhasselt.be

Stijn Vansummeren
UHasselt
Data Science Institute, ACSL
Belgium
stijn.vansummeren@uhasselt.be

ABSTRACT

Data ingestion validation, the task of certifying the quality of continuously collected data, is crucial to ensure trustworthiness of analytics insights. A widely used approach for validating data quality is to specify, either manually or automatically, so-called data unit tests that check whether data quality metrics lie within expected bounds. We employ conditional unit tests based on conditional metrics (CMs) that compute data quality signals over specific parts of the ingestion data and therefore allow for a fine-grained detection of errors. A violated conditional unit test specifies a set of erroneous tuples in a natural way: the subrelation that its CM refers to. Unfortunately, the downside of their fine-grained nature is that violating unit tests are often correlated: a single error in an ingestion batch may cause multiple tests (each referring to different parts of the batch) to fail. The key challenge is therefore to untangle this correlation and filter out the most relevant violated conditional unit tests, i.e., tests that identify a core set of erroneous tuples *and* act as an explanation for the errors. We present CM-Explorer, a system that supports data stewards in quickly finding the most relevant violated conditional unit tests. The system consists of three components: (1) a graph explorer for visualizing the correlation structure of the violated unit tests; (2) a relation explorer for browsing the tuples selected by conditional unit tests; and, (3) a history explorer to get insight why conditional unit tests are violated. In this paper, we discuss these components and present the different scenarios that we make available for the demonstration.

PVLDB Reference Format:
Niels Bylois, Frank Neven, and Stijn Vansummeren. CM-Explorer: Dissecting Data Ingestion Problems. PVLDB, 16(12): 3958 - 3961, 2023.
doi:10.14778/3611540.3611595

1 INTRODUCTION

Modern data analytics pipelines continuously collect and ingest new data. Validating the quality of collected data at ingestion time is crucial in such pipelines, for a number of reasons. First, and foremost, the quality of the derived insights, and the decisions driven by them, depend directly on the quality of the collected data [8]. Second, as more and more of the data analysis process is automated, small errors in source data risk propagating to later data consumers (such as machine learning models), which may themselves act as data sources in other processing pipelines—thereby potentially magnifying the error [3]. Finally, data errors may even cause data processing pipelines to crash (e.g., because of null pointer exceptions due to missing data). Machine learning platforms are therefore including explicit data validation components into their pipelines [1, 2, 5].

In recognition of the importance of data quality validation in modern analytics pipelines, several tools have been proposed to aid in automatic validation [3, 7, 8]. Broadly speaking, these tools allow specification, either manually or automatically, of so-called *data unit tests*. When a new batch of data is to be ingested, the registered tests are executed to gauge the batch's quality where failing tests highlight data quality problems. The tests themselves entail computing certain metrics on the data batch (e.g., the minimum or maximum value appearing in a numerical column or the number of distinct elements appearing in a column) and checking that these fall within an expected range.

Unfortunately, these tools suffer from two limitations. First, the data unit tests that they support are based on *global* metrics: metrics that are computed on the entire data batch, or on an entire column in the batch. As the following example shows, they hence only provide *coarse-grained* signals of data quality and are unable to detect *fine-grained* errors, i.e., errors that occur only in a specific (potentially small) part of the batch. Batches with fine-grained errors hence go unnoticed. Second, even when a data unit test signals that a batch has a data quality problem, it does not provide a principled method to identify the part of the batch that is responsible for a test's failure. As such, either the entire batch must be discarded or a human expert must manually identify and subsequently remove or correct the erroneous tuples—which is time-consuming and labor-intensive.

Example 1.1. A public railway company has equipped all of its trains with measurement sensors that record the train's arrival and departure time at each train station. By comparing these times with the time schedule, the train's software computes the corresponding delays (if any). At the end of each day, the measurements of all trains are collected, and ingested in the railway company's data lake. The delays are used to identify hotspot routes, as well as computation of service quality indicators to the government. Train 5437 runs daily from Hasselt (Belgium) to Blankenberge (Belgium). This route is notorious for the delays that it incurs when it passes through the busy Brussels railway stations. As such, train 5437 normally reports non-zero delay. Due to a hardware malfunction on March 15, however, it consistently reports zero delay for this train.

The metrics used by state-of-the-art tools are unable to detect this error. Indeed: because zero delay is *not* an uncommon value when considering the entire ingestion batch (some trains run on

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611595

time), metrics such as min(delay), max(delay), and avg(delay) will not consider zero delay as an anomaly.

It is important to observe that, even if one of the global metrics signals a data quality problem, it is not clear which train or set of trains in the batch cause the problem. For instance, if the unit tests based on min(delay) or max(delay) signal an anomaly then of course we can easily identify the erroneous trains: simply compute the trains whose delay value is below (or above) the expected minimum (resp. maximum) value. However, if a unit test based on avg(delay) signals a problem, then it is unclear how to identify the trains that caused the delay to deviate. □

We have developed an approach for data quality validation that is not only capable of detecting the *fine-grained errors* illustrated above, but also helps in *identifying* responsible erroneous tuples [4]. Our methodology focuses on the setting where a data pipeline regularly ingests batches of external data. As the main technical vehicle underlying our methodology, we introduce *conditional metrics* (CM for short). In contrast to a global metric, a conditional metric only computes its value on a specified subset of tuples in the ingestion batch. A CM m is of the following form

$$m := \mu(Y \mid X = x) \qquad (\dagger)$$

where μ is a metric like MIN, MAX, SUM, MAXDIGITS, ...; Y and X are attributes of the relation under consideration and x is a domain value for X. We refer to (X, x) as the entity in m. The semantics is as follows: m computes the metric μ over the column Y for the subrelation consisting of all tuples where X equals the value x. In the railway example above, for instance, the conditional metric

$$\textsc{avg}(\mathit{Delay} \mid \mathit{Train} = 5437)$$

computes the average delay of train 5437 in the batch, as opposed to the global metric AVG(*Delay*) which computes the average delay of all trains. A conditional unit test then is simply a CM with an associated function that returns true for admissible values and false otherwise. Because of their ability to calculate values for specific entities (e.g., *Train* = 5437), conditional data unit tests may hence detect data quality issues at a finer level of granularity than data unit tests based on global metrics.

Our methodology consists of two phases: (i) a unit test discovery phase and (ii) a monitoring and error identification phase, as schematically illustrated in Figure 1. In the *unit test discovery phase*, we are given a sequence $\overline{R}$ of previously ingested batches and our objective is to automatically derive a set Θ of CM-based data unit tests from $\overline{R}$ such that a yet-to-be-ingested batch B can be considered to be of acceptable quality if it passes all tests in Θ. Specifically, like [7], we assume that the batches in $\overline{R}$ are themselves of acceptable data quality—which is reasonable because they have already been ingested successfully. Under this assumption, we may derive CM-based data unit tests from $\overline{R}$ by considering the set of all possible CMs, and deriving, for each such CM m, a classifier that allows to distinguish expected values of m from anomalies. If m produces an anomaly on a yet-to-be-ingested batch B, then B will be flagged as having a quality issue. To derive the classifier based on the values of m observed in $\overline{R}$, in principle any anomaly detection method [6] may be used. For illustration purposes, Figure 1 mentions some simple anomaly detection methods that produce classifiers that

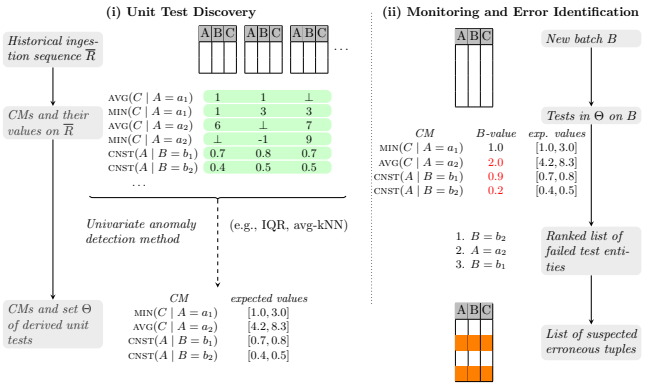


Figure 1: Overview of methodology: (i) *discovery phase* **to derive data unit tests based on conditional metrics; (ii)** *monitoring and error identification phase* **where each new batch is validated and erroneous tuples are identified.**

can be summarized as an expected range of values, where values outside the range are anomalies. Whatever method is chosen, we require that the derived classifier is consistent with our assumption that $\overline{R}$ is of reasonable quality, which implies that the values of m observed in $\overline{R}$ are without significant anomalies.

In the *monitoring and error identification phase*, we take the set Θ of data unit tests derived in the discovery phase, and use this to validate each new ingestion batch B. If all tests in Θ succeed on B then B is deemed to be of acceptable quality. When at least one test in Θ rejects B then our objective is to identify the tuples in B with suspected errors. Every conditional unit test specifies a set of erroneous tuples in a natural way: the subrelation of B that its CM refers to. In our railway example, if a unit test with CM AVG(*Delay* | *Train* = 5437) hence fails, all tuples with *Train* = 5437 could be flagged. Simply flagging *all* the tuples of violated unit tests selects too much, however (i.e., it results in high recall but very low precision). The reason is that violating tests are often correlated: a single error in B may cause multiple tests (each selecting different subrelations) to fail. The key challenge in the monitoring phase, therefore, lies in *ranking* the violated unit tests according to relevance, and from this ranked list of tests *filter* a list of suspected erroneous tuples for further inspection. We have developed multiple metrics to rank and filter violated unit tests in [4].

We illustrate the challenges in analyzing violated unit tests by means of an example. For this, we introduce the bipartite entity-tuple graph as a means to represent the correlation structure of the violated unit tests. This graph consists of two sets of nodes: all entities mentioned in at least one of the violated unit tests and every tuple that is selected by at least one of them. Furthermore, there is an edge from an entity to a tuple when that tuple is selected by the entity. Recall the scenario outlined in Example 1.1, where due to a hardware malfunction train 5437 consistently reports zero delay. Consider the to-be-ingested batch B shown in Figure 2 as well as the failed unit tests ϕ_1, ϕ_2, and ϕ_3 listed there. The entity-tuple graphs is displayed as well. We note that for the sake of this example, we have added an extra attribute *Kind* that lists the kind of service that the train offers. Unit test ϕ_1 fails because the average delay of train

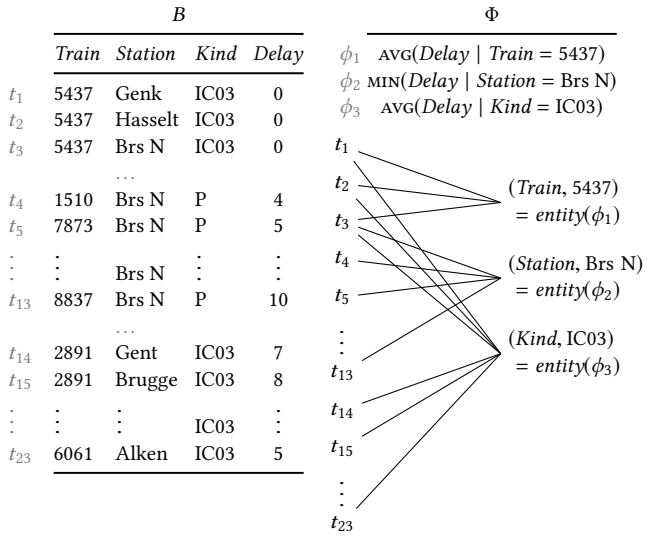


Figure 2: Illustration of a to-be-ingested batch B, failed unit tests Φ, and the corresponding entity-tuple graph.

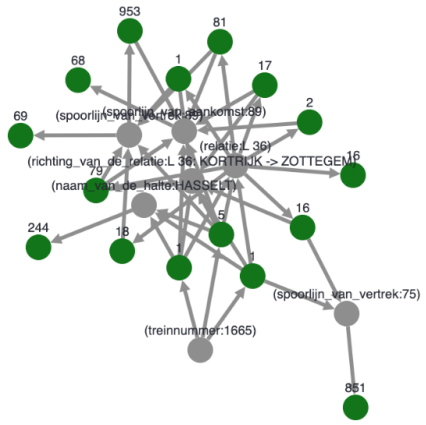


Figure 3: Entity-tuple graph explorer example for a single component for all *flagged* entities.

5437 is now zero, which is unexpected given the historical sequence. However, in this example, the zero delay of train 5437 also causes the minimum delay in station Brussels North (Brs N) to become zero, which is also unexpected, causing ϕ_2 to fail. Similarly, it also causes the average delay of route kind IC03 to become unexpected, failing ϕ_3. The root cause here is the zero delay of train 5437, and ideally we would hence like to filter out ϕ_1 that selects only the tuples t_1–t_3 of train 5437 as suspected erroneous tuples.

We present CM-Explorer, an interactive system that focuses on the monitoring and error identification phase of the methodology described above, and that allows to interactively investigate the observed data quality problems. Using CM-Explorer, a data steward may easily inspect the correlation structure of the violated unit tests via a graph-based representation to identify the most relevant ones for selecting a set of erroneous tuples and serving as an explanation for them. We next describe the components of CM-Explorer that support this (in Section 2), and discuss concrete demonstration scenarios in Section 3.

2 OVERVIEW OF CM-EXPLORER

The CM-Explorer consists of three components: the graph explorer, the CM history explorer, and the relation explorer. The workflow supported by CM-Explorer is as follows. A data steward may first utilise the graph explorer to filter the failed conditional unit tests by selecting tests with high scoring entities and removing tests with entities that refer to column names that are not considered to be important. In a second step, the data steward can zoom into individual entities using the history and relational explorer, to further reduce the conditional unit tests until only those of interest remain.

2.1 Graph explorer

An example of a real-world entity-tuple graph, as inspectable in the graph explorer, is displayed in Figure 3. The graph explorer always shows a contracted version of the entity-tuple graph, where tuple nodes are combined into a single node when they are connected to the same set of entity nodes. Contracted tuple nodes are colored green and carry as label the number of tuples that they represent. Entity nodes are colored grey. The graph explorer supports the data steward in analyzing the entity-tuple graph through a number of filters:

- Filtering on score: As explained above, we developed a number of metrics to rank entities and each of those can be used to filter for high scoring entities (and the associated CMs). When a particular measure is chosen, the score of each entity can be displayed.
- Filtering on entity column name: Consider the conditional metric AVG($Delay$ | $Train$ = 5437). Then $Train$ refers to the entity column name and $Delay$ to the column name the aggregate is applied to. The data steward can filter out specific column names that she regards as uninteresting thereby reducing the size of the entity-tuple graph.

To further support interactive exploration, the graph explorer component allows to remove specific entities from the graph, that, for instance, have been inspected by the components described in the following sections, and for which it has been established that they do not refer to erroneous tuples and can be safely disregarded.

2.2 History explorer

When an entity is selected in the graph explorer, the history view shows all the data points from previous batches in blue, with the lower and upper bounds highlighted in red (see Figure 4a for an example). The value on the current batch (that reported an error) is shown in green. When there is no value for a specific batch in the historical data, this is shown as a gap in the graph. The user can hover over a specific data point to inspect the exact value for that batch. The user can hence explore in more detail why specific

Fast and Scalable Data Transfer Across Data Systems

HARALAMPOS GAVRIILIDIS, BIFOLD and Technische Universität Berlin, Germany
KAUSTUBH BEEDKAR, Indian Institute of Technology Delhi, India
MATTHIAS BOEHM, BIFOLD and Technische Universität Berlin, Germany
VOLKER MARKL, BIFOLD, Technische Universität Berlin, and DFKI GmbH, Germany

Fast and scalable data transfer is crucial in today's decentralized data ecosystems and data-driven applications. Example use cases include transferring data from operational systems to consolidated data warehouse environments, or from relational database systems to data lakes for exploratory data analysis or ML model training. Traditional data transfer approaches rely on efficient point-to-point connectors or general middleware with generic intermediate data representations. Physical environments (e.g., on-premise, cloud, or consumer nodes) also have become increasingly heterogeneous. Existing work still struggles to achieve both, fast and scalable data transfer as well as generality in terms of heterogeneous systems and environments. Hence, in this paper, we introduce a holistic data transfer framework. Our XDBC framework splits the data transfer pipeline into logical components and provides a wide variety of physical implementations for these components. This design allows a seamless integration of different systems as well as the automatic optimizations of data transfer configurations according to workload and environment characteristics. Our evaluation shows that XDBC outperforms state-of-the-art generic data transfer tools by up to 5×, while being on par with specialized approaches.

CCS Concepts: • **Information systems** → *Data federation tools*; **Extraction, transformation and loading**; **Data exchange**.

Additional Key Words and Phrases: Data Transfer, Data Loading Optimization, Data System Interoperability

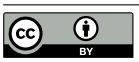
ACM Reference Format:
Haralampos Gavriilidis, Kaustubh Beedkar, Matthias Boehm, and Volker Markl. 2025. Fast and Scalable Data Transfer Across Data Systems. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 157 (June 2025), 28 pages. https://doi.org/10.1145/3725294

1 Introduction

The typical IT landscape of today's organizations comprises various decentralized data ecosystems and many data-driven applications [29, 34]. Over the last decades, there was a trend towards specialized data systems to improve performance and functionality [84, 85]. Apart from processing systems, modern data lakes allow storing raw data in various formats. Since data is distributed across these heterogeneous data representations and systems, efficient data transfer mechanisms are crucial for seamless integration and data analysis.

A Need for Data Transfer: There is a strong need for efficient data transfer because data movement and conversion is often resource-intensive and time-consuming. Accordingly, there is work on eliminating unnecessary data transfer. For example, in-database machine learning aims to conduct training and especially inference inside the DBMS [22, 49, 77] to avoid costly exports from the DBMS.

Authors' Contact Information: Haralampos Gavriilidis, BIFOLD and Technische Universität Berlin, Germany, gavriilidis@tu-berlin.de; Kaustubh Beedkar, Indian Institute of Technology Delhi, India, kbeedkar@cse.iitd.ac.in; Matthias Boehm, BIFOLD and Technische Universität Berlin, Germany, matthias.boehm@tu-berlin.de; Volker Markl, BIFOLD, Technische Universität Berlin, and DFKI GmbH, Germany, volker.markl@tu-berlin.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/6-ART157
https://doi.org/10.1145/3725294

Similarly, IoT systems push computation down to the edge to reduce unreliable network transfers [31, 92]. However, there are many use cases where data must be moved, either to leverage specialized tools, to integrate systems, or to consolidate data, e.g., data science applications [90], federated systems [26, 41, 44, 76, 78], extraction-transformation loading (ETL) flows [80], or data replication [56]. Overall, many modern data-driven applications would benefit from efficient data transfer mechanisms.

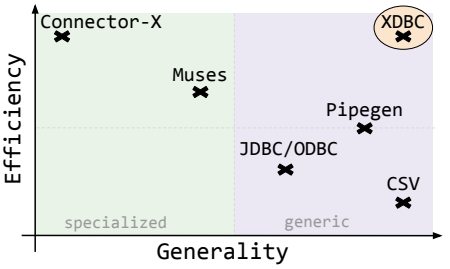


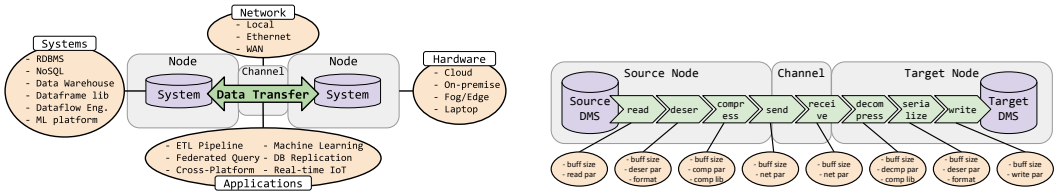
Fig. 1. Data Transfer Approaches.

Data Transfer Challenges: Designing an efficient data transfer mechanism is challenging due to the heterogeneity of data systems, particularly in terms of interfaces, data formats, and varying environment characteristics. Data transfer across heterogeneous systems is commonly realized through custom (i.e., specialized) point-to-point connectors, or generic solutions such as integration middleware or generic JDBC/ODBC drivers [6]. Figure 1 shows this trade-off space of efficiency versus generality. First, specialized connectors provide good performance but lack generality because they are implemented for specific pairs of systems. Second, generic adapters like JDBC/ODBC are primarily designed for lightweight DBMS access and suffer from poor performance [74]. Additionally, data transfer involves multiple steps (e.g., reading, deserializing, compressing, and transmitting), which must be tuned for data, system, and network characteristics. For example, compression may improve the transfer in unreliable consumer-cloud networks, but yield slowdowns in a cloud environment. The complexity of heterogeneous systems and environments renders the optimization of data transfer onerous. Hence, there is a need for a data transfer mechanism that yields robust performance, is easy to use, and general enough for data transfer across heterogeneous systems.

Limitations of Existing Work: Despite work on optimizing ETL [80, 81] and integration flows [28], the literature on efficient data transfer is relatively sparse. Raasveldt et al. analyzed DBMS client protocols and found that JDBC-based protocols are unsuitable for transferring large amounts of data due to their row-based nature and excessive metadata in each protocol buffer [74]. Pipegen generates pipes from existing CSV I/O unit tests in DBMSes, and redirects text I/O to compressed binary communication [48]. Zigzag joins optimize data transfers during joins of data warehouse tables and files in distributed file-system via Bloom filters [86]. Connector-X improves the loading of DBMS tables into pandas dataframes through parallelism and efficient data handling [90]. While these approaches optimize specific aspects of data transfer, there is a lack of a holistic data transfer framework that adapts to different physical environments and supports heterogeneous systems.

Contributions: To address the data transfer challenges and limitations of existing work, we introduce XDBC as a holistic data transfer framework. XDBC offers reader and writer interfaces that can be implemented for arbitrary data systems, eliminating the need for multiple connectors per system pair. XDBC's architecture comprises decoupled components (for read, deserialize, compress, send, receive, decompress, serialize, and write) as well as different physical implementations, which facilitate a flexible configuration and tuning for different environments and data characteristics. In order to free users from the need of manual tuning, XDBC's heuristic optimizer automatically selects effective configurations for given environments. Our detailed technical contributions are:

- **XDBC Architecture:** We describe the architecture and runtime of XDBC in Section 3. The simple yet very effective design allows seamlessly integrating existing data systems as well as tuning the individual data transfer components.



(a) Data Transfer Topologies and Applications.

(b) Data Transfer Anatomy: Configurable Pipeline.

Fig. 2. Data Transfer Across Heterogeneous Systems

- **XDBC Optimizer:** We introduce a novel optimizer for automatically tuning the data transfer configurations in different environments in Section 4. This heuristic optimizer selects parallelization and transfer strategies, and adapts configurations to monitored runtime statistics.
- **Experiments:** Finally, we present the results of an extensive experimental evaluation in Section 5. We compare XDBC with a variety of baselines on real and synthetic datasets as well as different simulated network topologies.

2 Background

In this section, we introduce the necessary background of data transfer, guided by a running example. Data transfer is crucial for many applications, and existing solutions fall short for generic data transfers across heterogeneous systems in different environments.

2.1 Motivation and Running Example

As shown in Figure 2a, data transfer has many different dimensions: different applications run on different data systems and infrastructures (hardware and networking). In the following, we introduce a running example and describe the state-of-the-art data transfer.

Example 1 (Data Science Use Case). *Assume a data-science application, where data practitioners need to load a large dataset from a DBMS, such as PostgreSQL, into a data science framework, such as Python pandas, to pre-process the data, run exploratory data analysis, and train various ML models. The data transfer between the DBMS and pandas may occur in different environments with very different characteristics. First, pandas may run on a local laptop, connected via a slow wide-area-network to the cloud. Second, the data practitioner may run pandas in a remote, cloud-hosted, environment, such as Google Colab, which is better connected to the DBMS.*

Generic Data Transfer: To load the data from the DBMS into pandas, one can utilize generic ODBC-based connectors, such as the widely-used `turbodbc` [12] and `sql.reader` [8]. Such plug-and-play connectors are very practical: users install the drivers and issue queries against the DBMS. The row-based results are then put into a collection of objects such as a pandas dataframe. Most ODBC (and JDBC) drivers only offer limited tuning knobs to optimize the transfer. The most commonly used tuning parameter is the number of rows to fetch in each round-trip to the DBMS. To further optimize performance, users could manually issue multiple queries—that partition the table by primary key—and then assemble the partitions. Existing data loading tools (e.g., Connector-X [90] and Spark JDBC [11]) also follow this approach and generate the partitioning queries automatically. However, the used row format—ideal for row-oriented DBMSes—has shown to be inefficient due to metadata bloat of in-flight buffers and limited compressibility [74].

Specialized Data Transfer: An alternative solution is a custom (i.e., specialized) connector that bypasses the generic JDBC/ODBC driver and establishes a direct connection to the DBMS. This approach allows more fine-grained control and direct conversion from the DBMS-internal



Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes

Simran Arora
Stanford University
simarora@stanford.edu

Brandon Yang
Stanford University
bcyang@stanford.edu*

Sabri Eyuboglu
Stanford University
eyuboglu@stanford.edu*

Avanika Narayan
Stanford University
avanikan@stanford.edu

Andrew Hojel
Stanford University
ahojel@stanford.edu

Immanuel Trummer
Cornell University
itrummer@cornell.edu

Christopher Ré
Stanford University
chrismre@stanford.edu

ABSTRACT

A long standing goal in the data management community is developing systems that input documents and output queryable tables without user effort. Given the sheer variety of potential documents, state-of-the art systems make simplifying assumptions and use domain specific training. In this work, we ask whether we can maintain generality by using the in-context learning abilities of large language models (LLMs). We propose and evaluate Evaporate, a prototype system powered by LLMs. We identify two strategies for implementing this system: prompt the LLM to directly extract values from documents or prompt the LLM to synthesize code that performs the extraction. Our evaluations show a cost-quality tradeoff between these two approaches. Code synthesis is cheap, but far less accurate than directly processing each document with the LLM. To improve quality while maintaining low cost, we propose an extended implementation, Evaporate-Code+, which achieves better quality than direct extraction. Our insight is to generate many candidate functions and ensemble their extractions using weak supervision. Evaporate-Code+ outperforms the state-of-the art systems using a *sublinear* pass over the documents with the LLM. This equates to a 110× reduction in the number of documents the LLM needs to process across our 16 real-world evaluation settings.

PVLDB Reference Format:
Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. PVLDB, 17(2): 92 - 105, 2023.
doi:10.14778/3626292.3626294
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/HazyResearch/evaporate.

1 INTRODUCTION

Organizations often seek insights trapped in heterogeneous data lakes (*e.g.* the web, corporate data lakes, and electronic health records) [10, 26, 54]. In their raw form, these data sources cannot easily support analytical queries. A long standing goal of the data management community is to develop systems that automatically convert heterogeneous data lakes into queryable, structured tables [12, 15, 47, 66, inter alia.]. In this work, we investigate whether recent large language models can help address this problem.

We study systems that take as **input** heterogeneous documents (*e.g.* HTML webpages, PDFs, text) and **output** a tabular, structured view of the documents. These systems must identify the schema and perform extraction to populate the table.

> Example 1. Medical researchers frequently use data spanning electronic health records (EHR), clinical trials, knowledge sources (e.g. PubMed), and FDA reports to understand and monitor patients and treatments [8]. Consider the large collection of **FDA 510(k)** reviews for premarket medical devices, which have been the subject of multiple studies [64, 68]. Our objective is to output a table that automatically structures the attributes that are distributed in the ~20-page **PDFs**, for instance the `device classification`, `predicate device code`, and `indications for use`.

Systems designed to tackle this problem must balance a three-way tradeoff between **cost** (data lakes may hold millions of documents), **quality** (output tables should be able to accurately support an analyst's queries), and **generality** (different data lakes have different document types and structure). See Section 2 for a formal task definition and further discussion of this tradeoff.

Given the range of formats, attributes, and domains across documents, prior systems rely on simplifying assumptions (e.g. handling one document format). The majority of works focus on structuring HTML [12, 15, 25], assuming the attributes and values are at specific positions in the HTML-DOM [22, 43, 44, 67]. For unstructured text, current approaches use linguistic tools (e.g., dependency parsers) to introduce structure [15, 25, 31, 48] and then apply heuristic rules over the resulting structure to extract information. The documents in Example 1 highlight the limitations of the prior approaches: they lack (e.g. HTML) structure and, consistent with recent evaluation efforts [67], we find the SoTA approaches for unstructured text perform poorly on long semi-structured PDFs (See [5]). Some systems assume there is a human-in-the-loop, labeling data and writing heuristic rules for extraction [53, 57], while others assume access to annotated training documents from the domain [22, 43, 44]. Researchers manually annotated the reports in Example 1 [64].

* Equal contribution.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 2 % ISSN 2150-8097.
doi:10.14778/3626292.3626294

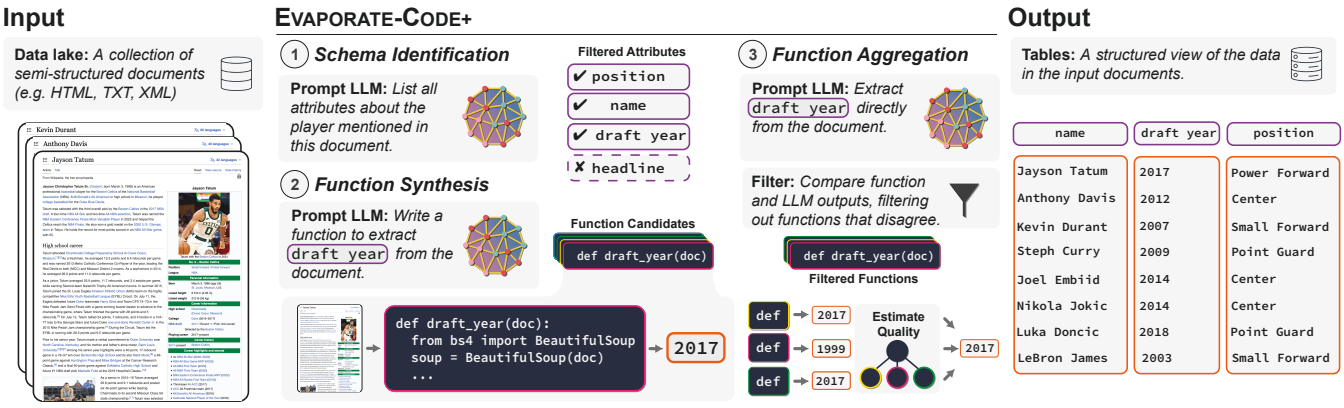


Figure 1: The user provides a collection of documents (e.g. NBA player bios) and EVAPORATE outputs a table by identifying attributes and populating columns. EVAPORATE avoids running expensive LLM inference on all documents by (1) synthesizing the key attributes from a small sample of documents and (2) synthesizing (e.g. Pythonic) functions that then are reused at scale to process documents. Because function quality is variable, EVAPORATE (3) applies an algorithm that generates many candidate functions and ensembles their extractions using weak supervision.

In this work, we explore whether we can improve generality by leveraging *large language models* (LLMs). An LLM is a deep learning model that is pretrained on broad data and can be adapted to diverse tasks, from machine translation to data wrangling [14, 46]. At inference time, the models take as input a natural language task description termed a *prompt* [11, 14] and generate a natural language response. See Section 2.3 for more background on LLMs.

EVAPORATE. (Section 3) We present EVAPORATE, a system that uses LLMs to produce structured views of semi-structured data lakes. Our evaluation spans 16 real-world settings from movie and university websites to *FDA 510(k)* reviews [22, 32, 34, 37, 43, 64, 68].

The user inputs a collection of documents and EVAPORATE automatically identifies the schema and performs extraction to populate the table. Our implementation requires *no customization, training, or human effort* to support the diverse evaluation settings. We propose two fundamental strategies for implementing this interface, identifying a tradeoff between their cost and quality:

(1) EVAPORATE-DIRECT (Figure 2) The LLM directly extracts values from documents.
(2) EVAPORATE-CODE (Figure 4) The LLM *synthesizes code* that is then applied to process documents at scale.

EVAPORATE-CODE is cheap, but underperforms EVAPORATE-DIRECT by 24.9% (13.8 F1 points) averaged across our evaluation settings. We thus seek a new code synthesis approach. We present EVAPORATE-CODE+, which achieves better quality than direct extraction. Our insight is to synthesize many code snippets for extraction and ensemble their outputs using weak supervision.

Direct Extraction (Section 3.1). Our first implementation, EVAPORATE-DIRECT, applies a *single prompt* (included in [5]) to each document in the input. The prompt instructs the LLM to both identify the schema and extract values. Remarkably, we find that in some settings, with a single prompt and no task specific modifications, performance is already competitive with state-of-the-art systems that rely on domain specific assumptions and training.

However, this implementation is very expensive. LLMs are optimized for interactive, human-in-the-loop applications (*e.g.* ChatGPT) [65], not high-throughput data processing tasks [56]. The number of tokens processed by an LLM in EVAPORATE-DIRECT grows *linearly* with the size of the data lake. As of March 2023, applying OpenAI's models to the 55 million Wikipedia articles would cost over $110k (`gpt-3.5`, $0.002/1k tokens) and $1.1M (`text-davinci-003`, $0.02/1k tokens) dollars [1, 49]. There are *billions* of webpages on the broader Internet [35] and the facts change over time. For instance, NBA players are added to Wikipedia, a player's `team` changes after trades, and the `points per game` metric changes after every game. Data processing is a *routine expense* (repeated by multiple data analysts), not a one-time cost [55].

Code Synthesis (Section 3.2). *Can we produce the structured table using a sublinear pass of the LLM over the documents?* We propose EVAPORATE-CODE, which splits the task into two sub-tasks: (1) identify the table schema and (2) extract values. This view allows us to exploit the distinct *redundancies* of each sub-task that occur when running LLM inference on every document:

(1) *Schema Generation.* In order to identify a schema, we only process a small sample of documents with the LLM. This succeeds because there is redundancy in the attributes mentioned across documents. For e.g., in Example 1, most reports mention a `predicate device name`.
(2) *Function Synthesis.* We prompt the LLM to synthesize (*e.g.* Pythonic) *functions*, that can be applied at scale across the documents. This works because of redundancy in the formatting of attribute-value pairs. For e.g., the FDA 510(k)s use the consistent format "`Predicate device name: k`".

The number of tokens processed by the LLM in EVAPORATE-CODE is *fixed* and does not grow with the size of the data lake (as illustrated in Figure 3), addressing the cost issues of EVAPORATE-DIRECT. However, the LLM synthesizes variable quality information extraction functions. The extractions are up to 14 points worse in Pair F1 score than those produced using EVAPORATE-DIRECT.

Code Synthesis + Aggregation. (Section 3.3) To improve quality while keeping costs low, we propose Evaporate-Code+. Studying the synthesized functions, we observe some only work for a narrow slice of documents, while others exhibit syntactic and logical errors. To reduce variance, we synthesize many candidate functions, then estimate their quality and aggregate their extractions using *weak supervision*. This builds on our work [4], which broadly applies weak supervision to prompting for the first time.

Weak supervision (WS) is a statistical framework for modeling and combining noisy sources with varied coverages without any labeled data [53, 62]. However, WS is typically applied over *human-generated* functions while our setting consists of *machine-generated* functions. This presents issues when attempting to apply existing WS tools. (1) WS theoretically assumes all noisy sources are better than random performance (50% accuracy), yet 40% of our generated functions are *below 25%* (Section 3.2). (2) WS attempts to deploy functions that achieve high quality on narrow slices of data (high precision), and allow the function to *abstain* on data external to the slice (low recall). While humans can express when functions should abstain, the machine-generated functions do not contain this logic. To handle the *open* WS setting, we introduce a new algorithm for ensembling the functions (Algorithm 1).

We summarize our overall contributions as follows.

(1) **Our system offers new capabilities for the long-studied structured view generation problem.** Existing systems require in-domain training and handle limited document formats (e.g. HTML [16, 22, 24, 43]). Evaporate requires no training and succeeds on different document formats (HTML, PDF, TXT) off-the-shelf. (Section 4).
(2) **We study a new tradeoff space between direct extraction and code synthesis for data tasks.** Evaporate *asymptotically* reduces the number of tokens that the LLM needs to process to generate the outputs. At 10k documents per evaluation setting, this amounts to a 110x cost reduction. Further, prior works using LLMs for data tasks require users to manually write prompts [46]. Evaporate is built with task-agnostic prompts that generalize across settings.
(3) **We present an algorithm and theoretical analysis for applying weak supervision to open-ended functions and extraction tasks.** Although Evaporate-Code is more efficient, Evaporate-Direct achieves significantly higher quality. Using our algorithm, Evaporate-Code+ outperforms Evaporate-Direct, which directly processes every document, by 10.1 F1 points (18%) (Table 3).
(4) **We extensively validate the system on 16 data settings from 5 domains and 3 data formats, and across 4 LLMs.** (1) Evaporate outperforms the SoTA *learned* baseline systems by 3.2 F1 points (6%) when generating tables (both schema generation and extraction) end-to-end, and 6.7 F1 points (10%) on the extraction step. (2) Evaporate-Code+ achieves a 10.1 F1 point increase over Evaporate-Direct, using text-davinci-003. (3) Across four unique LLMs we show the relative quality of Evaporate-Direct vs. Evaporate-Code+ remains consistent.

We define the problem in Section 2. We present Evaporate in Section 3, evaluations in Section 4, and related works in Section 5.

2 PRELIMINARIES

We first define the problem setting and system desiderata.

2.1 Problem Setting

We study the problem of constructing a structured view (*i.e.* database table) of a set of semi-structured documents (*e.g.* HTML, PDF, TXT). Formally, we define the problem as follows:

- **Input:** User provides a set of n semi-structured documents $D = \{d_1, d_2, ...d_n\}$ (*e.g.* A collection of FDA 510(k) reviews for premarket notification submission for medical devices).
- **Output:** System outputs a table defined by a set of attribute names $A = \{a_1, a_2, ...a_m\}$ (*e.g.* a_1 =`indications for use`, a_2=`classification`) and a set of n extracted records for $R = \{r_1, r_2, ...r_n\}$, one per document, where r_i is an m-tuple (*e.g.* r_1 = ("fracture", "x-ray")).

Unlike prior work which proposes systems that rely on manual labeling [57] or manual prompt tuning [46, 59], we aim to develop *automated* solutions, which require no user effort.

Measuring System Quality We compare the generated table (A, R) to a manually curated "ground-truth" table $(\hat{A}, \hat{R})$. The coverage of an attribute refers to the fraction of documents that include the attribute and its value. Following prior work, we prioritize attributes with high *coverage*, which tend to be useful for analysis [16, 18]. We measure agreement between the tables using Pair F1. For additional details on our evaluation setup, see Section 4.3.

2.2 System Desiderata

Current systems for producing structured views are limited in their generality, cost/flexibility, and quality/usability [16, 18, 48, 67]. Here we review the existing systems.

Generality. *The ideal system will generalize across document formats and domains, without manually engineered rules or task-specific training.* This is important because the input documents D could focus on any imaginable topic or use any file format [67]. Existing systems featurize documents by tagging the named entities (NER), dependency parse tree, and part-of-speech (POS), and train a model to predict whether a span of text is a useful fact [39]. Unfortunately, the performance of the parse, NER, and POS tags drastically degrade on semi-structured data (e.g. HTML elements) and longer sequences of text (i.e. full documents) [67]. We provide detailed error analysis [5]. A specialized class of systems focuses on processing semi-structured web HTML documents by leveraging the HTML DOM tree as features [13, 16, 22, 25, 44, inter alia.]. However, the systems thus do not support other document formats.

Cost. *The ideal system will enable users to manage a cost-coverage tradeoff, rather than requiring them to extract "all-or-nothing".* The existing systems are built to extract *all* possible facts in the documents, without prioritizing important attributes or allowing the user to influence what is extracted [21, 67]. Processing every line of every document can be expensive. To mitigate this, the user can define the attributes of interest then apply a closed IE system for extraction, however this requires upfront human effort. **Desiderata**: The ideal system will enable users to manage a cost-coverage tradeoff, rather than requiring them extract "all-or-nothing".



ReCG: Bottom-Up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework

Joohyung Yun
POSTECH
Pohang, Republic of Korea
jhyun@dblab.postch.ac.kr

Byungchul Tak*
Kyungpook National University
Daegu, Republic of Korea
bctak@knu.ac.kr

Wook-Shin Han*
Graduate School of AI
POSTECH, Republic of Korea
wshan@dblab.postech.ac.kr

ABSTRACT

The schemalessness, one of the major advantages of JSON representation format, comes with high penalties in querying and operations by denying various critical functions such as query optimizations, indexing, or data verification. There have been continuous efforts to develop an accurate JSON schema discovery algorithm from a bag of JSON documents. Unfortunately, existing schema discovery techniques, being top-down algorithms, face challenges from the lack of visibility into children nodes of JSON tree. With absence of the information about lower-level JSON elements, top-down algorithms need to employ assumptions and heuristics to decide the schema type of nodes. However, such static decisions are often violated in datasets which causes top-down algorithms to perform poorly. To overcome this, we propose an algorithm, called ReCG, that processes JSON documents in a bottom-up manner. It builds up schemas from leaf elements upward in the JSON document tree and, thus, can make more informed decisions of the schema node types. In addition, we adopt MDL (Minimum Description Length) principles systematically while building up the schemas to choose among candidate schemas the most concise yet accurate one with well-balanced generality. Evaluations show that our technique improves the recall and precision of found schemas by as high as 47%, resulting in 46% better F1 score while also performing 2.11× faster on average against the state-of-the-art.

PVLDB Reference Format:
Joohyung Yun, Byungchul Tak, and Wook-Shin Han. ReCG: Bottom-Up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework. PVLDB, 17(11): 3538 - 3550, 2024.
doi:10.14778/3681954.3682019

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/postechdblab/VLDB2024_ReCG.

1 INTRODUCTION

JSON (JavaScript Object Notation) is a widely used data representation format that has become de-facto standard for RESTful Web API's data exchanges [2, 23, 30, 44] and big data analytics [5, 10, 16, 27, 31, 40, 41]. Accordingly, efficient processing of massive volumes of JSON data and querying has become critical for services and operations. However, the efficiency of these operations is often hindered by the lack of schemas for the JSON documents at hand. Although the lack of strict schema enforcement of JSON format offers flexibility and rapid deployability, it instead exacerbates the data management costs by making it difficult to apply various operations [42]. Schemas are essential for many important use cases such as JSON validation [3, 34], JSON parsing acceleration [27, 31], migration of two data sources with different schemas [43], query formulation [15, 48], query optimizations [7, 9, 18, 33], fine-grained access control [42], JSON-to-relational data translation [13, 25, 46], and query answering [47]. Despite these practical benefits, JSON schemas are often nonexistent or unavailable to the users and, thus, have to be derived from a given set of JSON documents.

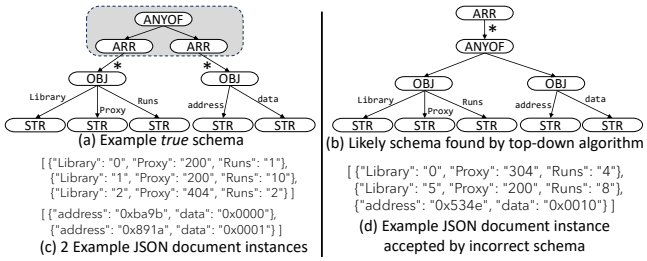


Figure 1: Illustration of top-down approach's limitation. It may lead to incorrect results (b) due to lack of visibility.

Due to the importance of JSON schemas, the JSON schema discovery (JSD) problem and topics related to the JSON schema formalization have been investigated [3, 4, 6, 11, 16, 21, 34, 38, 42]. Despite many efforts, deriving correct JSON schemas from JSON documents remains difficult due to several reasons. First, the ground truth schema set we want to derive may not exist in the first place for the given JSON documents. It is not uncommon that JSON documents are created without explicit schemas defined in advance. Users of JSON data prefer to create, load, and use the data quickly without the burden and delay from the schema construction task [32, 39]. The lack of a predefined schema also implies that the representation of objects in JSON can be inconsistent and has a high degree of variability [24]. Second, the derivation of a huge number of *correct* and *valid* schema sets is possible for a given JSON document set, but with varying degrees of generality. If the derived schemas are too general, the data validation becomes ineffective [38]. Moreover, the specific details of the schemas have to be eventually encoded into the query which increases the query processing overhead during the execution. On the other hand, if too specific, the size of the schema set increases and the schema handling costs rise significantly. A working solution to JSD problem should be able to find the schema with the right balance between representational simplicity and details within the vast search space of valid schemas.

*Corresponding authors
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3682019

A JSON schema can be viewed intuitively as having a tree-like hierarchical structure as shown in Figure 1 (a). The intermediate nodes in this tree structure are either objects or arrays that recursively hold other objects, arrays, or primitive types. The goal of the JSON schema discovery (JSD) problem is to learn this template structure from a given bag of JSON documents. Two example JSON documents are shown in Figure 1 (c). The top-down style of JSON schema discovery, adopted by the state-of-the-art algorithms [4, 6, 38], starts processing from the root and it proceeds down to the subsequent child levels repeatedly. However, this top-down approach faces the following nontrivial challenges. When a new node is encountered, it can be difficult to determine the correct node types (e.g., `ARR`, `OBJ`, `ANYOF`) since it has to be done without the knowledge of yet unseen lower-level nodes. This inevitably leads to the use of heuristics to make the best-effort decision of what the true type of the current node is. For example, at the shaded region of Figure 1 (a), it illustrates one possible case where the top-down algorithm failed to differentiate two types of objects and, thus, comes up with the incorrect (and overly general) schema as shown in (b) which accepts a mixture of objects. In addition, existing algorithms make a decision of node types rigidly even when there are other possible candidates at each decision point. This leaves little chance to rectify incorrect decisions made at the upper level when the processing reaches the descendant nodes where decision errors become evident.

To overcome the limitations of top-down approaches, we have developed a methodology, called `ReCG`, that employs three techniques. *First, we systematically explore candidate JSON schema sets comprehensively by incrementally generalizing them from specific schemas to the most general ones.* As Figure 2 illustrates, there can be schema sets with varying generality for the same JSON document sets at some point during the JSON schema discovery. We do not attempt to guess the correct one during processing as any choice has the potential to grow to be the true schema when schema discovery is completed. *Second, we employ the MDL (Minimum Description Length) metric to our JSON schema discovery problem as a metric that guides us to select the most promising candidate schema set during the navigation of the search space.* The MDL metric, used successfully in `XTRACT` [22] for computing DTD's information size, can be used here to compute the number of bits required to represent the schema and the data (JSON document) instances. We demonstrate in our evaluation its validity as a metric for assessing the goodness of JSON schema set. *Third, we design a novel bottom-up style JSON schema discovery algorithm* that departs from the existing approaches. We argue that the bottom-up style of processing is inherently better suited to the JSON schema discovery problem. At any intermediate node, the determination of a node type is better informed since the knowledge of descendant nodes is already acquired. Any two nodes of identical type can actually be disparate when their children and descendants are taken into account. By using this bottom-up style approach, we can expect to reduce the uncertainty in making decisions and the reliance on heuristics.

We have designed and implemented the prototype of our technique and evaluated it using 20 real-world JSON datasets. Our evaluation showed that `ReCG` delivered the 42.57 ~ 47.80% improved F1 score compared to the state-of-the-art technique. We observed this was due to 39.36 ~ 46.84% improvements in recall and 15.55 ~ 23.33% improvements in precision. In terms of the algorithm running time, `ReCG` outperformed the state-of-the-art by 2.11×.

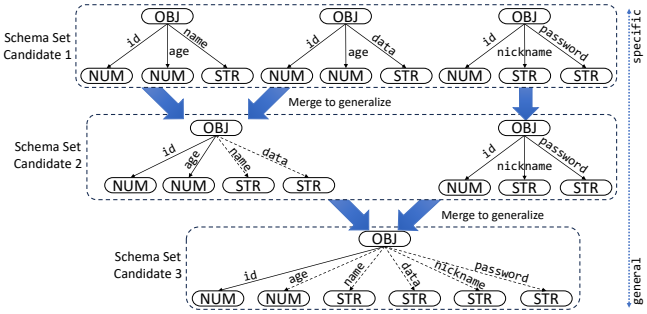


Figure 2: Schema sets with varying generality. More general schema sets are created by merging more specific sets.

J	::=	$P \mid O \mid A$	
P	::=	`null` | `true` | `false` | $n \mid s$	($n \in$ **Number**, $s \in$ **String**)
O	::=	$\{\ k_1 : J_1, \ldots, k_n : J_n\ \}$	($n \geq 0, i \neq j \implies k_i \neq k_j$)
A	::=	$[\ J_1, \ldots, J_n\ ]$	($n \geq 0$)

Figure 3: Grammar of JSON documents

We make the following contributions. First, we design a novel bottom-up algorithm, called `ReCG` for the JSON schema discovery problem that addresses major issues of existing algorithms. Accompanied by a technique for systematic search of schema sets by the specificity and the metric for assessing the goodness, our technique exhibits much-improved precision and recall against competing algorithms. Second, we provide supporting evidence that demonstrates the effectiveness of our technique using real-world data sets. Third, we make a prototype implementation of our technique as well as the dataset publicly available for the research community.

2 BACKGROUND

2.1 Definition

2.1.1 JSON document. We adopt the definition of JSON document introduced in the work by Baazizi et al. [6]. JSON documents are a set of strings that are derived from the grammar described in Figure 3. The derivation starts from a non-terminal J. It transitions to either P (primitive JSON document), O (object), or A (array). The non-terminal P further transitions to one of the following:

- *Number* type JSON document: C-like number. The set of these numbers is denoted as **Number**.
- *String* type JSON document: double-quoted sequence of zero or more Unicode characters. The set of such double-quoted sequences is denoted as **String**.
- *Boolean* type JSON document: `true` or `false`
- *Null* type JSON document: `null`

The non-terminal O is derived to an object, which is an unordered set of (key, JSON document) pairs. An object begins with '{' and ends with '}', and the pairs are separated by ','. We denote the pairs within an object as *key-value pairs*. A is derived to an array, which is an ordered sequence of JSON documents. We call the JSON documents within the sequence of array as *elements*. An array begins with '[' and ']', with elements separated by ','. The JSON document can be recursively defined. That is, JSON documents derived from J can recursively occur within objects and arrays.

2.1.2 JSON Schema. *JSON schema*s are a set of strings in the Backus-Naur form grammar, introduced by Pezoa et al. [34]. We adopt a subset of Pezoa et al.'s production rules since real-life schemas use only a limited subset of grammars, as introduced specifically by Spoth et al. [38]. The grammar of our interest is shown in Figure 4, and we denote a JSON schema as $\mathcal{S}$. In this grammar, non-terminals are expressed in *italic*, and the terminals are expressed in consolas.

2.1.3 Homogeneity and Heterogeneity of Object and Array Schemas. For object schemas, derivations from *homO* and *hetO* impose constraints on the key-value pairs of an object. Derivation from *homO* imposes objects to be *homogeneous*; keys that can be present within an object's key-value pairs are determined as *key*s. Also, the corresponding value for each *key* must be validated against the schema derived by *JS* (look $key : JS$ at the right-hand side of *prop*). *req* is used to list the keys that must be present in an object. Meanwhile, derivation from *hetO* imposes objects to be *heterogeneous*. The number of key-value pairs is unconstrained and the keys may be any random string. However, the values must be validated against the schema derived from *JS*. We name object schemas that have *homO* non-terminal derived as *homogeneous object schema*s, and *hetO* derived as *heterogeneous object schema*s. Object schemas with derivations from both *homO* and *hetO* are *composite object schema*s.

For array schemas, derivation from *homA* imposes arrays to be *homogeneous*; the number of elements within an array must be fixed, and each element at index i must be validated against the ith schema paired by the key "items". Whereas, derivation from *hetA* imposes arrays to be *heterogeneous*. The number of elements may vary for an array, but its elements should all be validated against the schema derived from the non-terminal *JS* paired by the key "items". We name array schemas that have *homA* non-terminal derived as *homogeneous array schema*s, and *hetA* derived as *heterogeneous array schemas*.

Lastly, derivation from *anyOf* is named as *anyOf* schema. It can have many schemas derived from *JS*s paired by the keyword "anyOf". An anyOf schema validates against a JSON document j if and only if one or more of its derivations can validate against j.

2.2 JSON Document Instance and Schema Representation

2.2.1 JSON Instance Tree. We model JSON documents as *JSON instance tree*s, which are node-typed, node-labeled and edge-labeled trees. Instance trees are extensions from *JSON tree*s introduced by Hutter and Augsten et al. [16]. We define an instance tree as $I = (V_I, E_I, \Psi_I, \Lambda_I, \Gamma_I)$ with the following notations.

- V_I is a set of nodes.
- $E_I \subseteq V_I \times V_I$ is a set of edges.
- $\Psi_I : V_I \rightarrow \{obj, arr, prm\}$ maps each node to its node type.
- $\Lambda_I : V_I \rightarrow$ **Number** $\cup$ **String** $\cup\{$true, false, null, $\epsilon\}$ maps a node to its label. ϵ means an empty label.
- $\Gamma_I : E_I \rightarrow$**String** $\cup\{\epsilon\}$ maps each edge to its edge label.

We transform a JSON document to a JSON tree as follows. Primitive JSON documents each become nodes with the type of *prm*. The nodes are further labeled with the values of JSON documents. Objects become nodes of type *obj* (labeled as ϵ) with the nodes of its values as children. The edge connecting an object node to its child is labeled with the key corresponding to the value of the child. An array is converted to an *arr* (labeled as ϵ) node with the i-th element becoming the i-th child subtree.

JS	::= { *SchCont* }	*prop*	::= , "properties": { (*key* : *JS*,)* }
SchCont	::= *strSch* | *numSch* | *boolSch* | *nullSch* | *objSch* | *arrSch* | *anyOf*	*req*	::= , "required": [(*key* ,)*]
		key	::= *s* (*s* ∈ **String**)
strSch	::= "type": "string"	*hetO*	::= , "additionalProperties": *JS*
numSch	::= "type": "number"	*arrSch*	::= "type": "array" (*homA* | *hetA*)
boolSch	::= "type": "boolean"	*homA*	::= , "items": *JS*
nullSch	::= "type": "null"	*hetA*	::= , "items": [(*JS*,)*]
objSch	::= "type": "object" *homO*? *hetO*?	*anyOf*	::= "anyOf": [(*JS*,)+]
homO	::= *prop req*?		

Figure 4: BNF grammar of JSON schemas.

2.2.2 Schema Tree. We also model a JSON schema as a tree, as it is expressed in recursive forms that can be captured well as a tree. A schema tree is a node-labeled, edge-labeled, and edge-typed tree defined as $s = (V_S, E_S, \Lambda_S, \Gamma_S, \Phi_S)$ with the following definition.

- V_S is a set of nodes. Each node represents a JSON schema.
- $E_S \subseteq V_S \times V_S$ is a set of edges.
- $\Lambda_S : V_S \rightarrow \{$ NUM, STR, BOOL, NULL, OBJ, ARR, ANYOF $\}$ maps each node to its node label which corresponds to the type of the schema.
- $\Gamma_S : E_S \rightarrow$ **String** $\cup\{ *, \epsilon \}$ maps each edge to its edge label. An edge label expresses either the derivation from *key* part in the right side of *prop*, or an "additionalProperties" of *hetO* part with a Kleene star $*$. Edge label $*$ indicates the heterogeneity of the schema node being the source node of that edge, while *key* indicates homogeneity.
- $\Phi_S : E_S \rightarrow \{$ Required, Optional, $\epsilon \}$ maps each edge to its type. The type of edge is only meaningful when $\Lambda_S(v)$ of its source node v is OBJ. It indicates whether a key of a homogeneous object schema is Required, or Optional. In our visualization, edges of Required type are expressed as solid lines, and edges of Optional are expressed as dotted lines.

For both instance trees and schema trees, we define an operator $v[l]$ for a node v and a label l.

$$v[l] = \begin{cases} v' & \text{if } \exists l \in \textbf{String},\ e = (v, v'),\ \Gamma(e) = l \\ v' & \text{else if } \exists l \in \mathbb{Z}^+,\ e = (v, v'),\ v' \text{ is } l^{th} \text{ child of } v \\ \epsilon & \text{otherwise} \end{cases} \quad (1)$$

It is an operator that returns v', the child subtree of v such that an edge $e = (v, v')$ exists which $\Gamma(e)$ is l, or the lth child subtree of v if $\Gamma(e)$ are all null. If such an edge does not exist, the return value is ϵ. We denote e_l as an edge of which $\Gamma(e) = l$. We denote $edgelabels(v)$ as a set of labels of v's outgoing edges.

2.3 Schema Assessment Metric: MDL Cost

The ideal set of JSON schemas should be general enough to accept all given positive JSON document instances and be specific enough to reject all negative JSON document instances. We need a method to quantify such quality of a set of JSON schemas to navigate through the search space. For this, we adopt the Minimum Description Length (MDL) principle [22, 35, 36] to our JSD problem.

The MDL principle, based on the information theory, offers a way to evaluate the adequacy of models inferred from a data set. It states that the best theory to infer a model from a set of data is the one that minimizes the sum of two components — *Schema Representation Cost* (SRC) and *Data Representation Cost* (DRC). The best model (i.e., set of JSON schemas in our problem settings) inferred